



Universidad
Zaragoza

Artificial Intelligence (62651)

Bayesian Machine Learning



Universidad
Zaragoza

Rubén Martínez Cantín

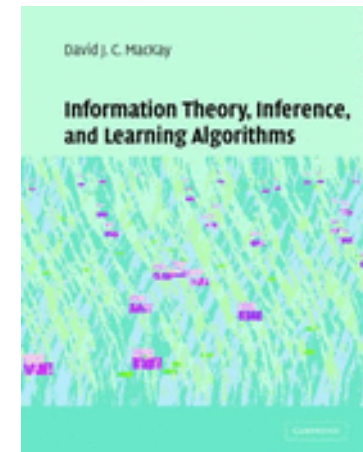
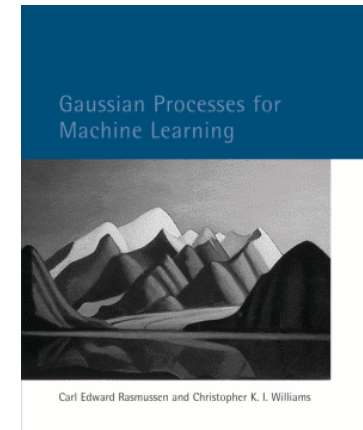
Dpto. Informática e Ingeniería de Sistemas.

Content

- **Bayesian learning**
- **Bayesian regression**
 - Bayesian linear regression
 - Gaussian parametric regression
- **Gaussian processes**
 - GPs for regression
 - Kernels
 - GPs beyond regression

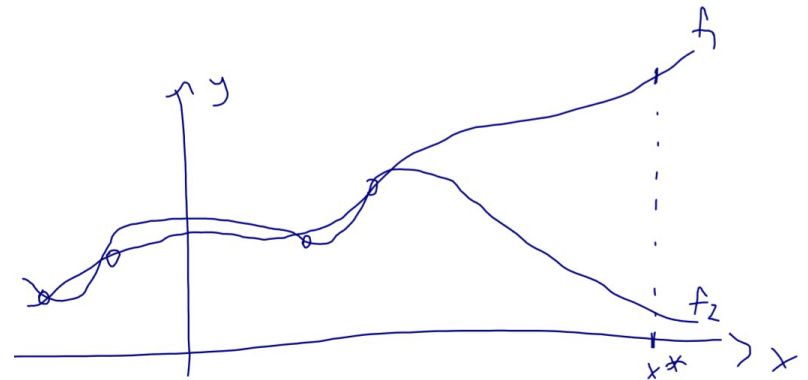
Complementary bibliography

- Carl Edward Rasmussen and Christopher K. I. Williams, Gaussian Processes for Machine Learning The MIT Press, 2006. ISBN 0-262-18253-X. <http://www.gaussianprocess.org/gpml/>
- MacKay, D.J., 2003. Information theory, inference and learning algorithms. Cambridge university press.
<http://www.inference.org.uk/mackay/itila/book.html>
- <https://distill.pub/2019/visual-exploration-gaussian-processes/>
- Online courses
 - Nando de Freitas, Machine Learning, <https://www.cs.ubc.ca/~nando/540-2013/lectures.html>
 - Phillip Henning, Probabilistic Machine Learning, <https://uni-tuebingen.de/en/180804>
- Python resources
 - In the labs: GPy, GPyOpt , scikit-learn.
 - Gpyflow, PyMC3...



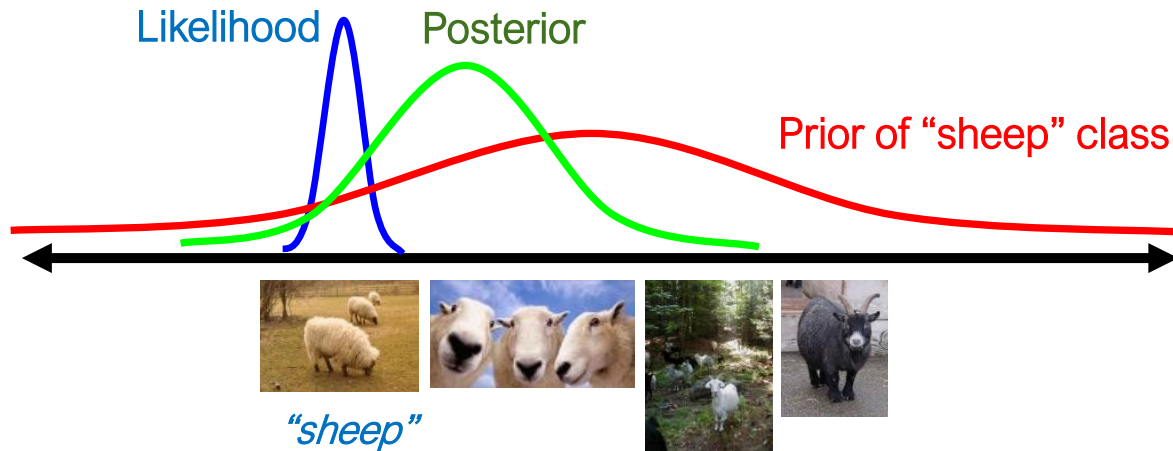
Example: Medical Diagnostics

- dAlbetes: an exciting new startup (not really)
 - claims to automatically diagnose diabetic retinopathy
 - accuracy 99% on their 4 train/test patients
 - engineer trained two deep learning systems to predict probability y
 - given input fundus image x .
- The engineer runs their system on your fundus image x^*
- Which model f_1, f_2 would you want the engineer to use for your diagnosis?
 - None of these! ('I don't know')
 - How do we say: 'I don't know'?



Bayesian learning

$$p(h | d) = \frac{p(d | h)p(h)}{\sum_{h' \in H} p(d | h')p(h')}$$



Basic statistics (reminder)

- Conditional probability

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

- Marginalization

$$P(A) = \int_{\mathcal{B}} P(A, B) dB$$

- Bayes Rule

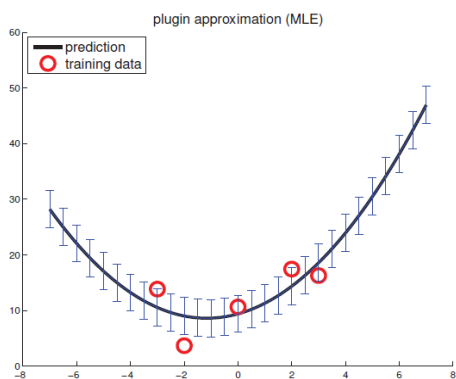
$$P(A|B) = \frac{P(B|A)P(A)}{\int_{\mathcal{A}} P(B|A)P(A)dA}$$

- Expectation

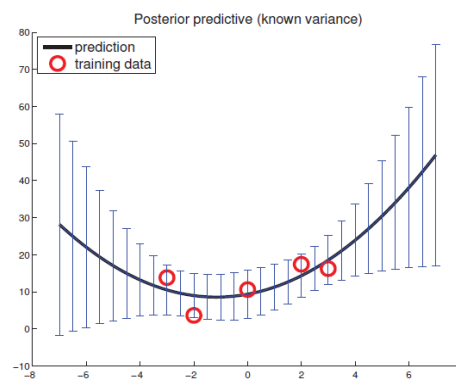
$$\mathbb{E}_{P(A)} f(A) = \int_{\mathcal{A}} f(A)P(A)dA$$

Bayesian regression vs ML/MAP

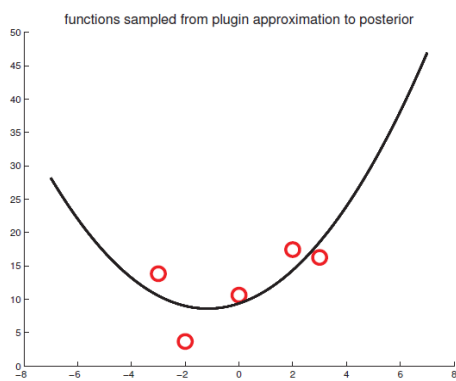
- In ML/MAP the result was a set of parameters $\hat{w}$
- The Bayesian approach computes the posterior $p(w|data)$



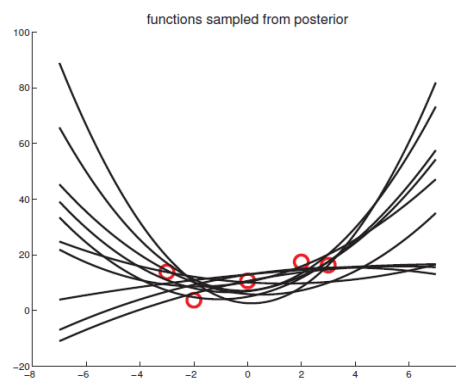
(a)



(b)



(c)



(d)

Bayesian learning for model parameters

- Step 0: Specify a **parametric model** with parameters w
- Step 1: Given data, $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, write down the expression for the likelihood:

$$p(D|w)$$

- Step 2: Specify a prior (conjugate, informative...):

$$p(w)$$

- Step 3: Compute the posterior:

$$p(w|D) = \frac{p(D|w)p(w)}{p(D)}$$

Bayesian linear regression

- The input/output relation is linear in the model parameters

$$y = x^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \quad \text{one data point}$$

$$y = Xw + \epsilon I_n \quad \epsilon \sim N(0, \sigma^2) \quad \text{dataset (size } n)$$

- What is the likelihood?

$$p(y|x, w) = N(y | x^T w, \sigma^2) \quad \text{one data point}$$

$$p(y|X, w) = N(y | Xw, \sigma^2 I_n) \quad \text{joint distribution dataset}$$

- What is the prior?

- For a linear-Gaussian likelihood model, the conjugate prior is a Gaussian.
- The posterior is also Gaussian $\rightarrow$ Closed-form, easy, allows recurrence.

$$p(w) = N(w | w_0, \Sigma_0)$$

Multivariate Gaussians are great!

- Represented by just 2 sets of parameters

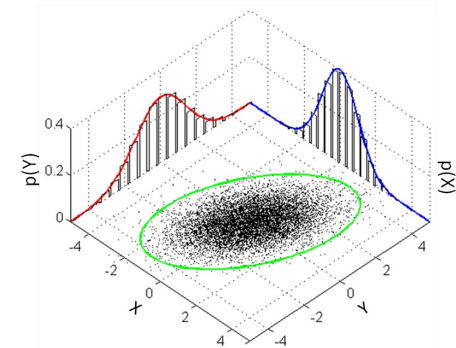
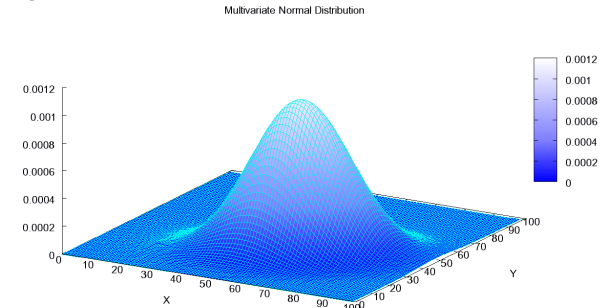
- Mean vector
- Covariance matrix (**SPD**)

- Important: Closed-form integral. Easy!

- Everything is Gaussian

- **Iff** $p(x,y)$ is Gaussian:

- $p(x)$ and $p(y)$ are Gaussian.
- $p(x|y)$ and $p(y|x)$ are Gaussian.
- Linear: $a * p(x) + b * p(y)$ is a Gaussian.
- Product: $p(x)*p(y)$ is a Gaussian:
- Convolution: $p(x + y)$ is a Gaussian



Images: Wikipedia

Multivariate Gaussians

- Theorem 4.3.1 (Murphy's book) $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\mu, \Sigma)$

$$\mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}, \quad \Lambda = \Sigma^{-1} = \begin{pmatrix} \Lambda_{11} & \Lambda_{12} \\ \Lambda_{21} & \Lambda_{22} \end{pmatrix}$$

Linearity

$$p(Ax_1 + Bx_2) = p(x|A\mu_1 + B\mu_2, A\Sigma_1A^T + B\Sigma_2B^T)$$

Marginal

$$p(\mathbf{x}_1) = \mathcal{N}(\mathbf{x}_1|\mu_1, \Sigma_{11})$$

$$p(\mathbf{x}_2) = \mathcal{N}(\mathbf{x}_2|\mu_2, \Sigma_{22})$$

Conditional

$$p(\mathbf{x}_1|\mathbf{x}_2) = \mathcal{N}(\mathbf{x}_1|\mu_{1|2}, \Sigma_{1|2})$$

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(\mathbf{x}_2 - \mu_2)$$

$$= \mu_1 - \Lambda_{11}^{-1}\Lambda_{12}(\mathbf{x}_2 - \mu_2)$$

$$= \Sigma_{1|2}(\Lambda_{11}\mu_1 - \Lambda_{12}(\mathbf{x}_2 - \mu_2))$$

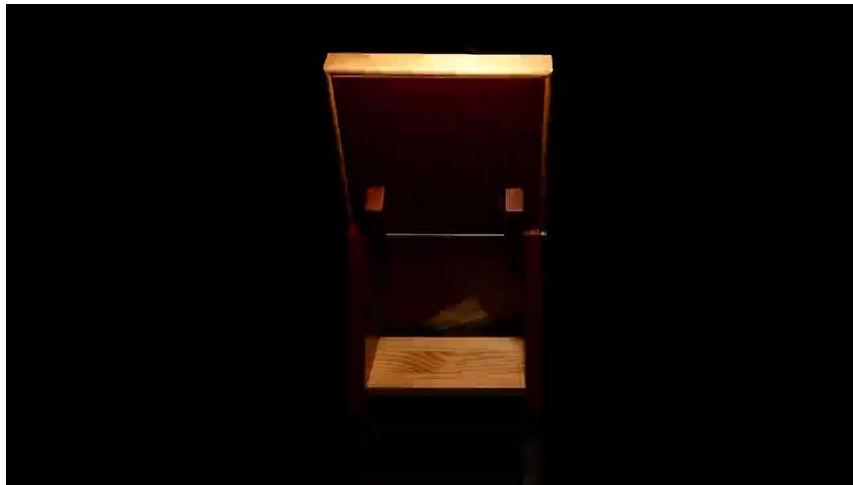
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21} = \Lambda_{11}^{-1}$$

Are Gaussian useful in practice?

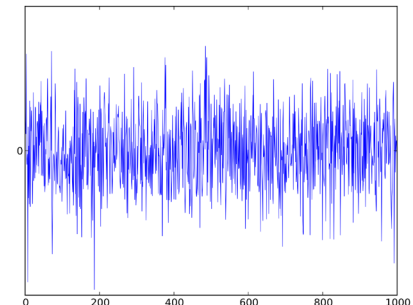
- “Everything ends up being Gaussian...”

- Central limit theorem

“... the sum of a number of independent and identically distributed random variables (with finite variances) will tend to a normal distribution, as the number of variables grows.”



- Noise is usually Gaussian



Bayesian linear regression (cont.)

- Likelihood: $p(\mathbf{y}|\mathbf{X}, \mathbf{w}) = N(\mathbf{y} | \mathbf{X}\mathbf{w}, \sigma^2 I_n)$

- Prior (general case): $p(\mathbf{w}) = N(\mathbf{w}|\mathbf{w}_0, \Sigma_0)$

- Posterior (see Murphy2012 Sec. 4.4.3):

$$p(\mathbf{w}|\mathbf{y}, \mathbf{X}) \propto N(\mathbf{y} | \mathbf{X}\mathbf{w}, \sigma^2 I_n) N(\mathbf{w}|\mathbf{w}_0, \Sigma_0) = N(\mathbf{w}|\mathbf{w}_n, \Sigma_n)$$

$$\mathbf{w}_n = \Sigma_n \Sigma_0^{-1} \mathbf{w}_0 + \frac{1}{\sigma^2} \Sigma_n X^T \mathbf{y}$$

$$\Sigma_n^{-1} = \Sigma_0^{-1} + \frac{1}{\sigma^2} X^T X$$

- If we use an isotropic Gaussian prior: $p(\mathbf{w}) = N(\mathbf{w}|0, \tau^2 I_d)$ then

$$\mathbf{w}_n = \frac{1}{\sigma^2} \Sigma_n X^T \mathbf{y} = (\lambda I_d + X^T X)^{-1} X^T \mathbf{y}$$

with: $\lambda = \frac{\sigma^2}{\tau^2}$

...which is again **ridge regression**!

Predictions in Bayesian regression

- Now that we have a distribution over parameters, we can integrate the parameters and get a **predictive posterior**.

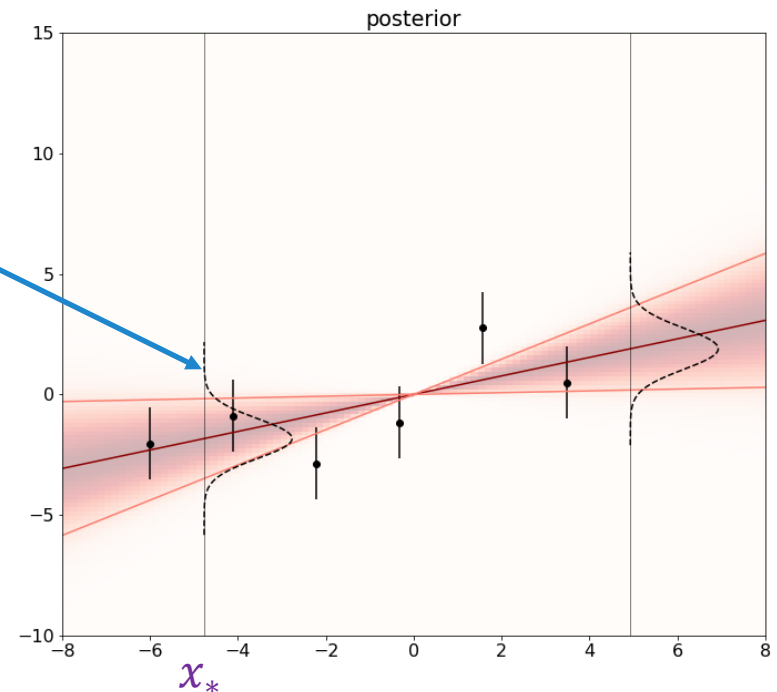
$$p(f_* | \mathbf{x}_*, \mathbf{X}, \mathbf{y}) = \int \underbrace{p(f_* | \mathbf{x}_*, \mathbf{w})}_{\text{likelihood of new point}} \underbrace{p(\mathbf{w} | \mathbf{X}, \mathbf{y})}_{\text{model posterior}} d\mathbf{w}$$

$$f_* \sim N(\mu(\mathbf{x}_*), \sigma_n^2(\mathbf{x}_*))$$

Remember

$$\mathbf{y} = \mathbf{x}^T \mathbf{w} + \epsilon$$

$$\begin{aligned}\mu(\mathbf{x}_*) &= \mathbf{x}_*^T \mathbf{w}_n \\ \sigma_n^2(\mathbf{x}_*) &= \sigma^2 + \mathbf{x}_*^T \Sigma_n \mathbf{x}_*\end{aligned}$$



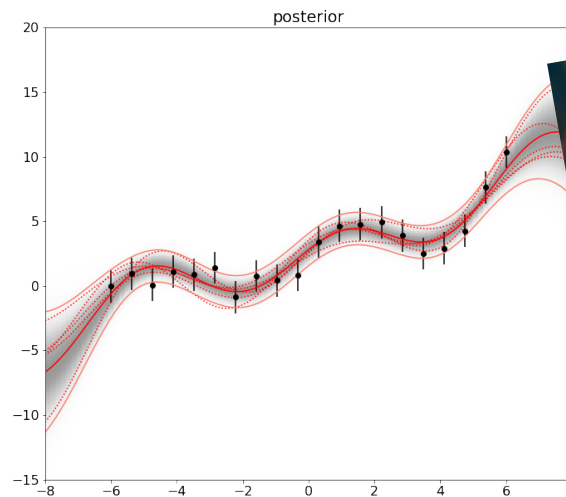
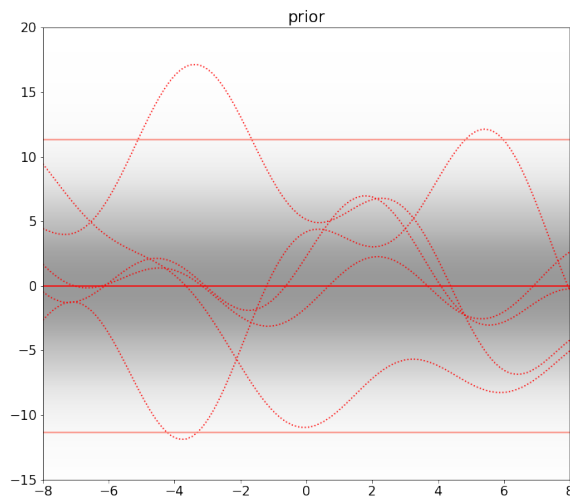
Parametric regression in a feature space

- If we use a **feature space** to project the **inputs** $\phi(x)$

$$y = \phi(x)^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \quad \text{one data point } \{x, y\}$$

$$y = \Phi_X^T w + \epsilon I_n \quad \text{dataset } \{X, y\} \text{ (size } n\text{)}$$

- The **output** is nonlinear, but the problem is still linear in the **weights**!



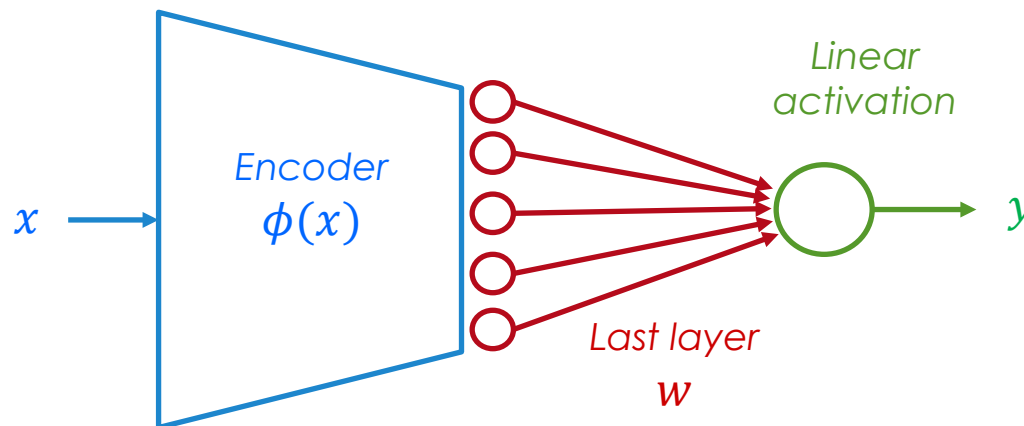
Parametric regression in a feature space

- If we use a **feature space** to project the **inputs** $\phi(x)$

$$y = \phi(x)^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \quad \text{one data point } \{x, y\}$$

$$y = \Phi_X^T w + \epsilon I_n \quad \text{dataset } \{X, y\} \text{ (size } n\text{)}$$

- This is also a neural network!



Parametric regression in a feature space

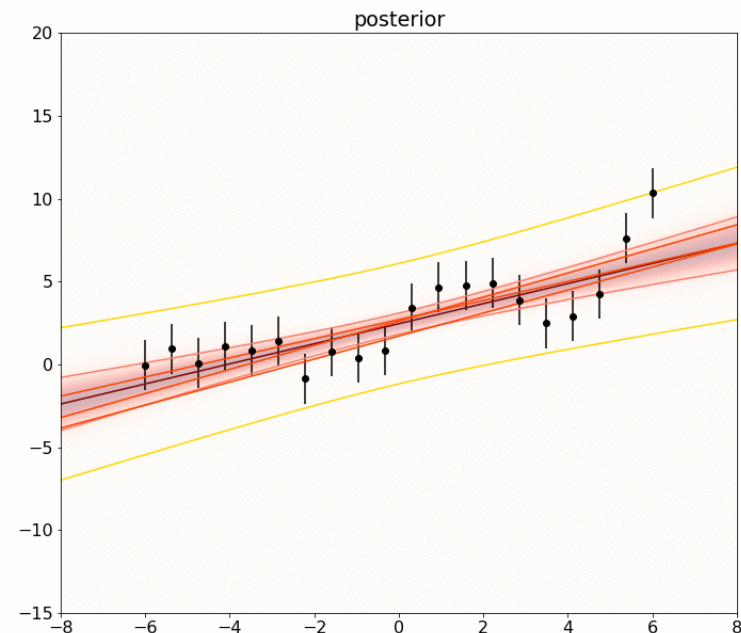
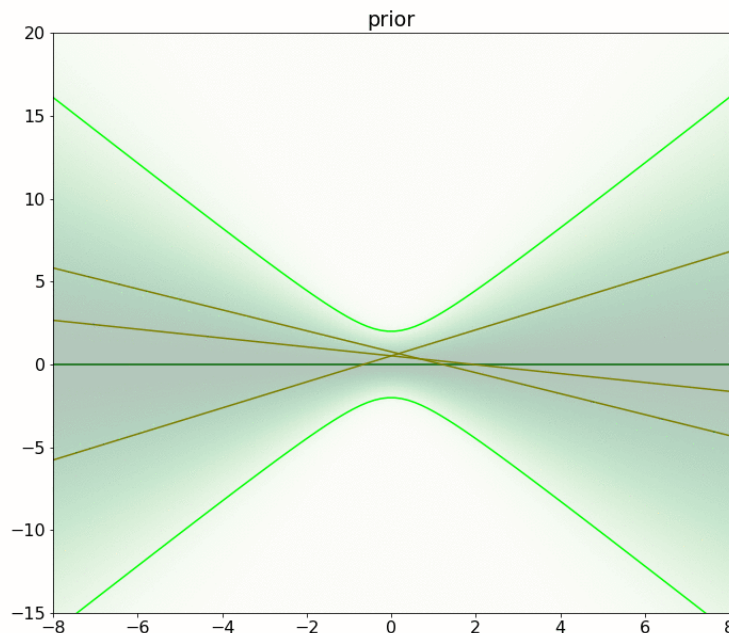
- Linear with a feature space $\phi(x)$

$$y = \phi(x)^T w + \epsilon \quad \epsilon \sim N(0, \sigma^2) \quad \text{one data point}$$

$$y = \Phi_X^T w + \epsilon I_n \quad \text{dataset (size } n\text{)}$$

- For example: $\phi(x) = (1, x)$

$$y = w_1 + w_2 x + \epsilon$$



Parametric regression

■ Priors:

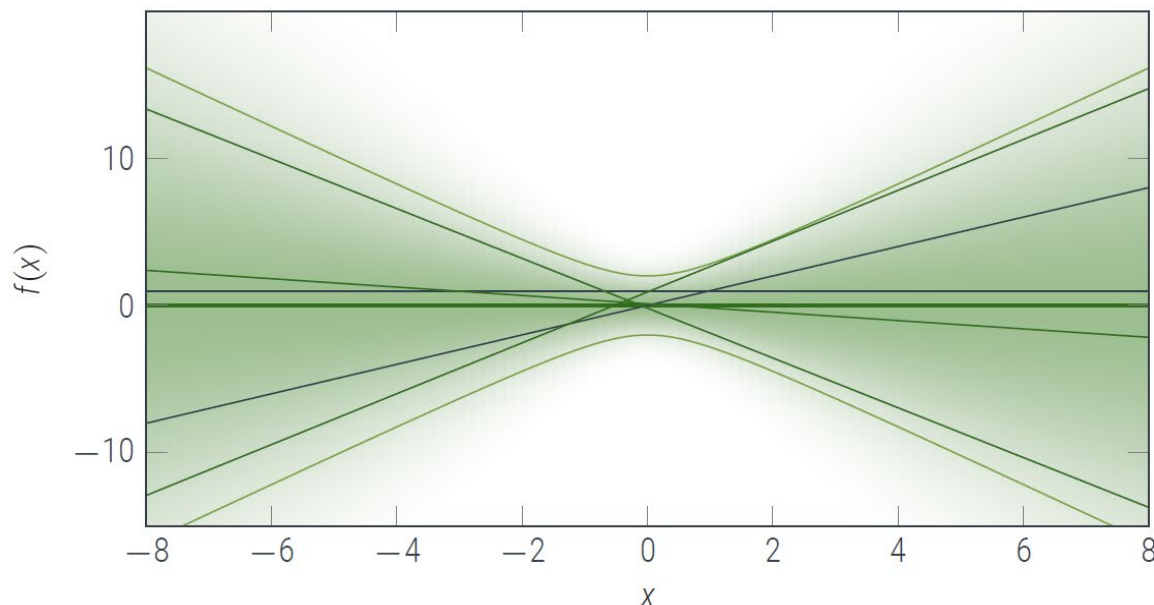
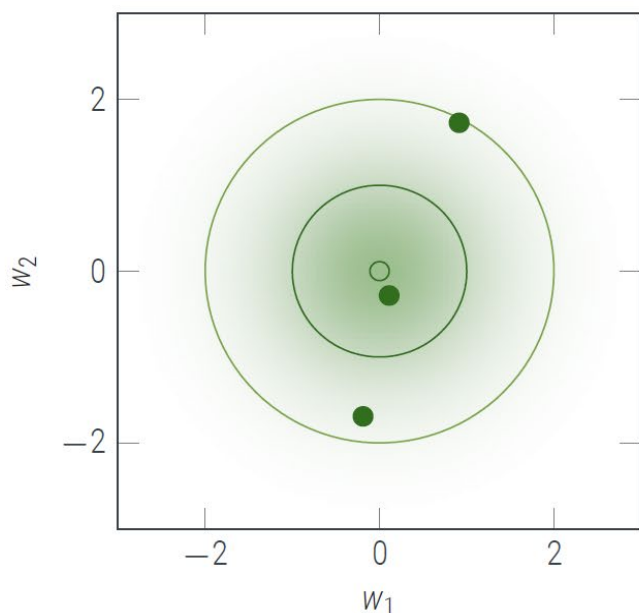
$$f(x) = \Phi_X^T w$$

■ Weight prior:

$$p(w) = N(w|0, \Sigma)$$

■ Predictive prior:

$$p(f_*) = N(f_*|0, \phi_X^T \Sigma \phi_X)$$



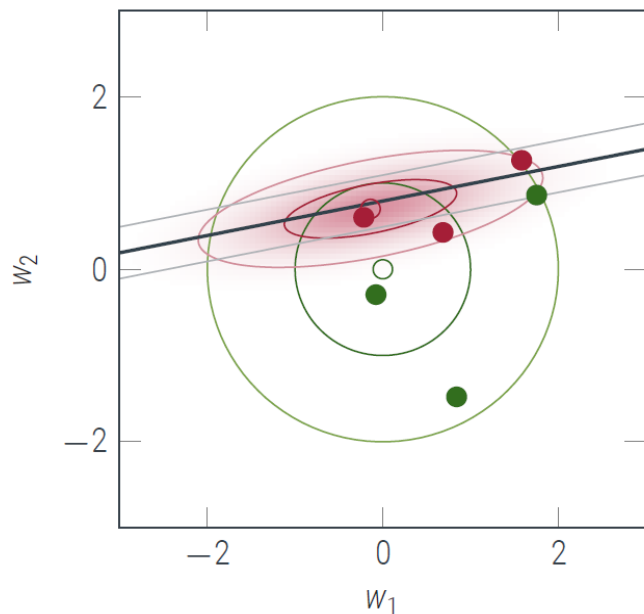
Parametric regression

■ Prior vs Likelihood vs Posterior

■ Weight space

$$p(w)$$

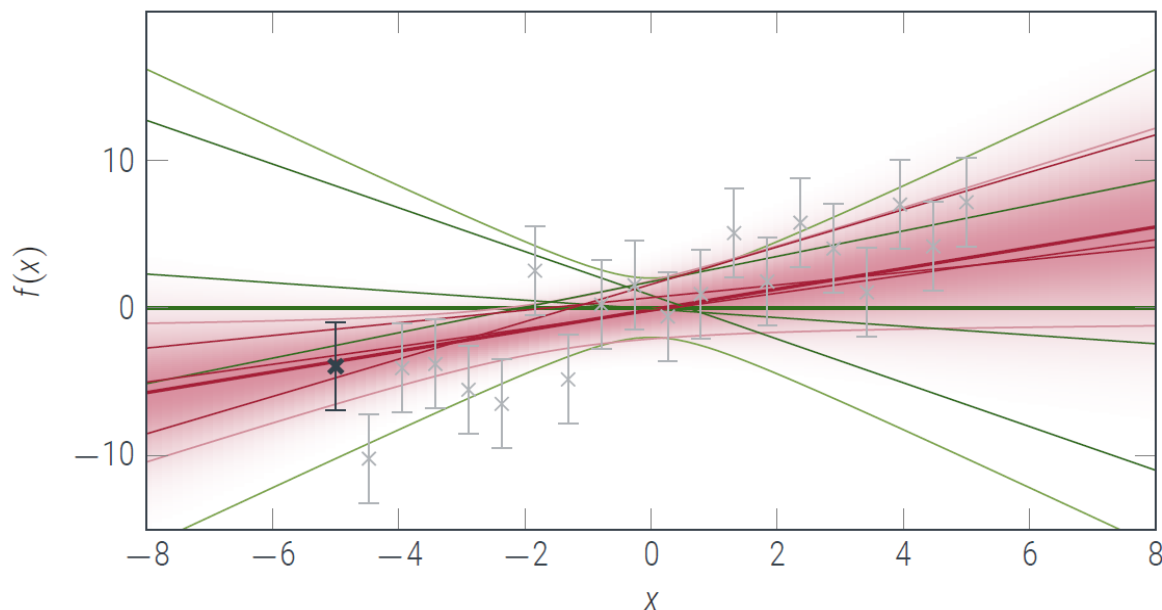
$$p(w|X, y)$$



■ Function (predictive) space:

$$p(f_*)$$

$$p(f_*|X, y)$$



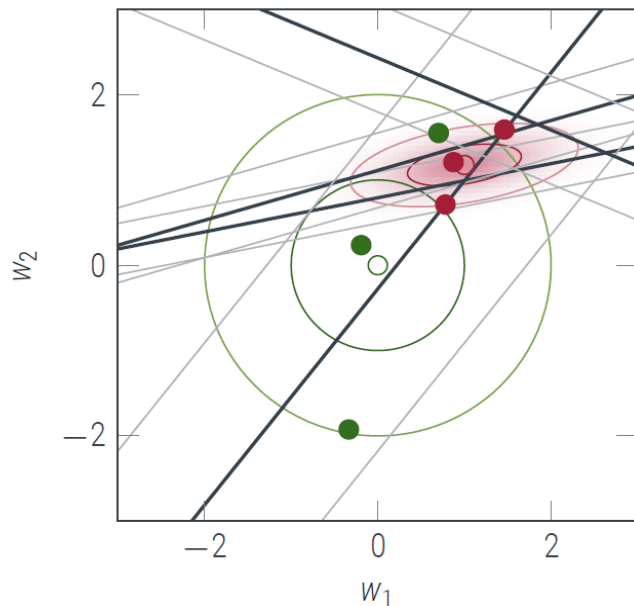
Parametric regression

■ Prior vs Likelihood vs Posterior

■ Weight space

$p(w)$

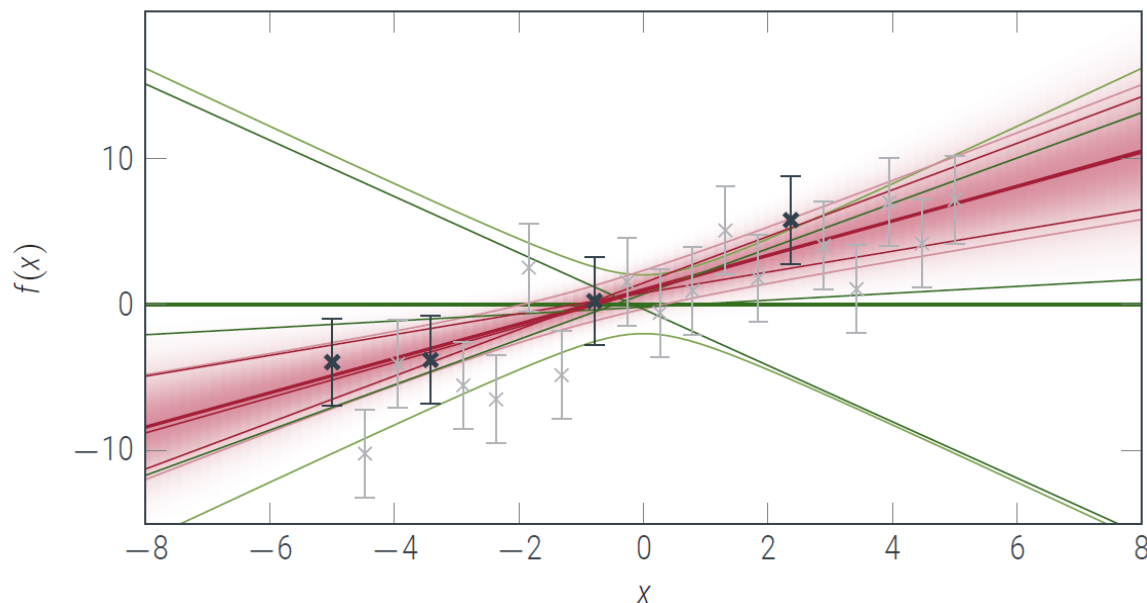
$p(w|X, y)$



■ Function (predictive) space:

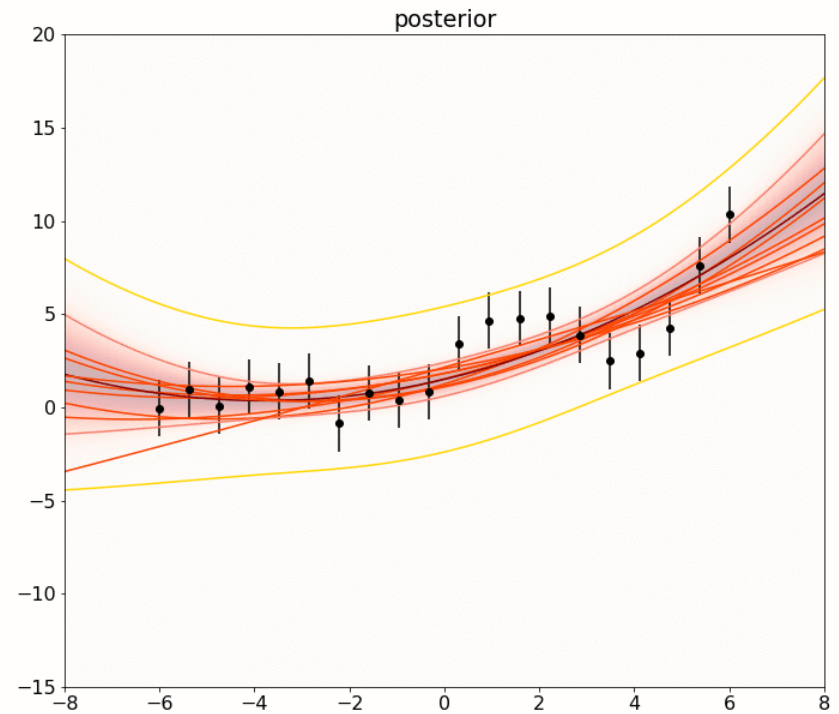
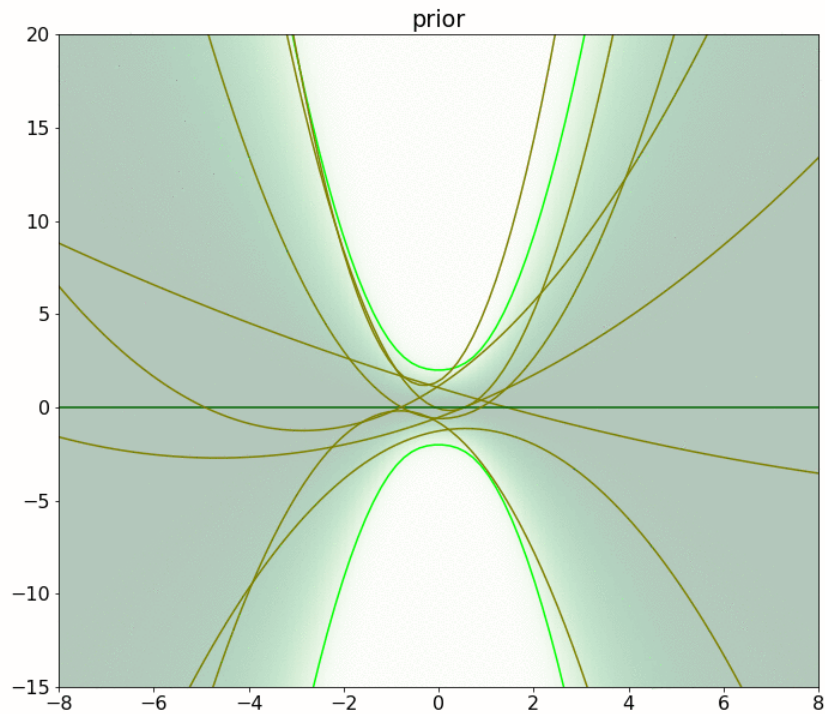
$p(f_*)$

$p(f_*|X, y)$



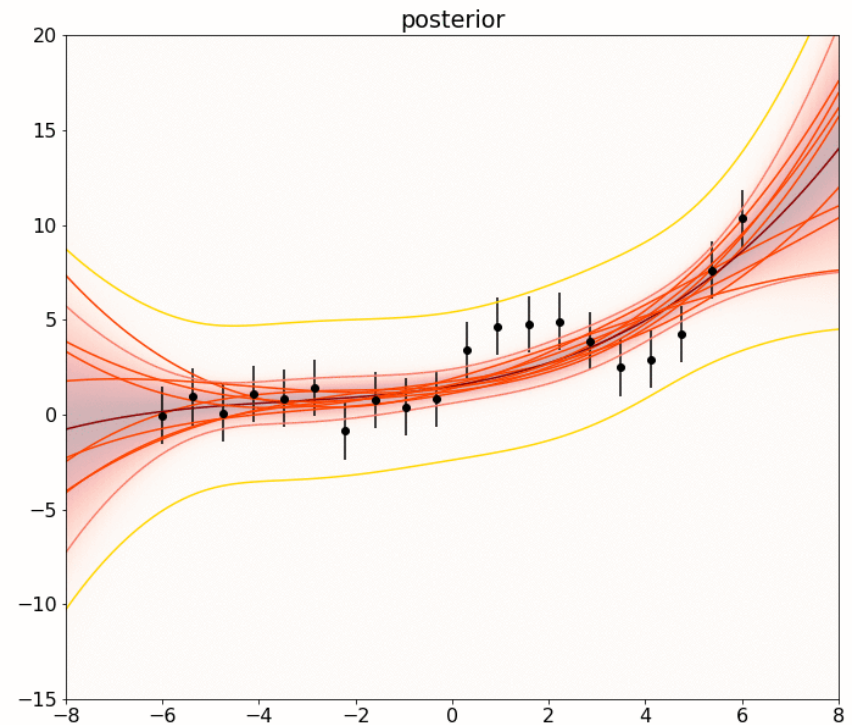
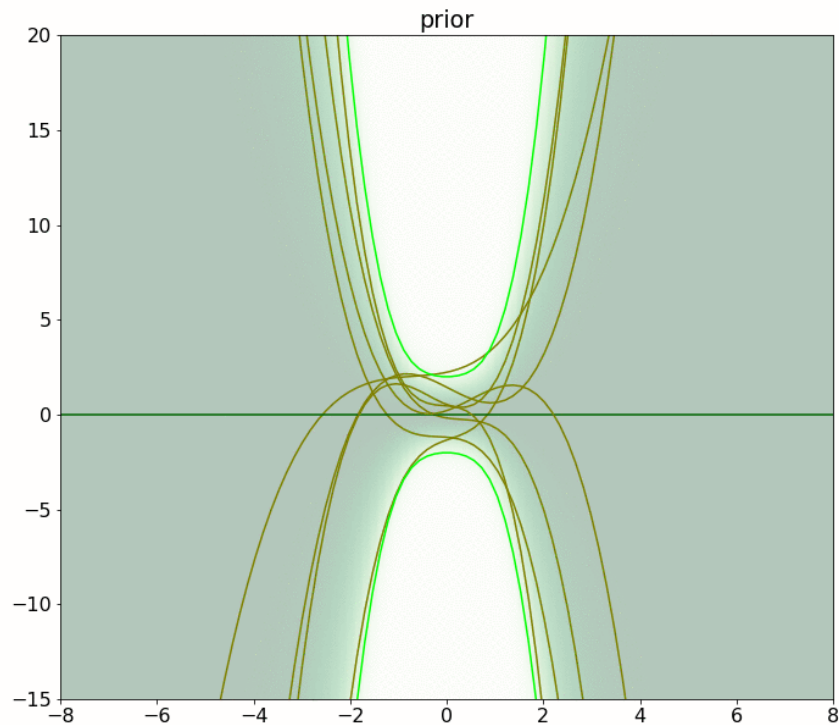
Polynomial features

■ $\phi(x) = [1, x, x^2]$



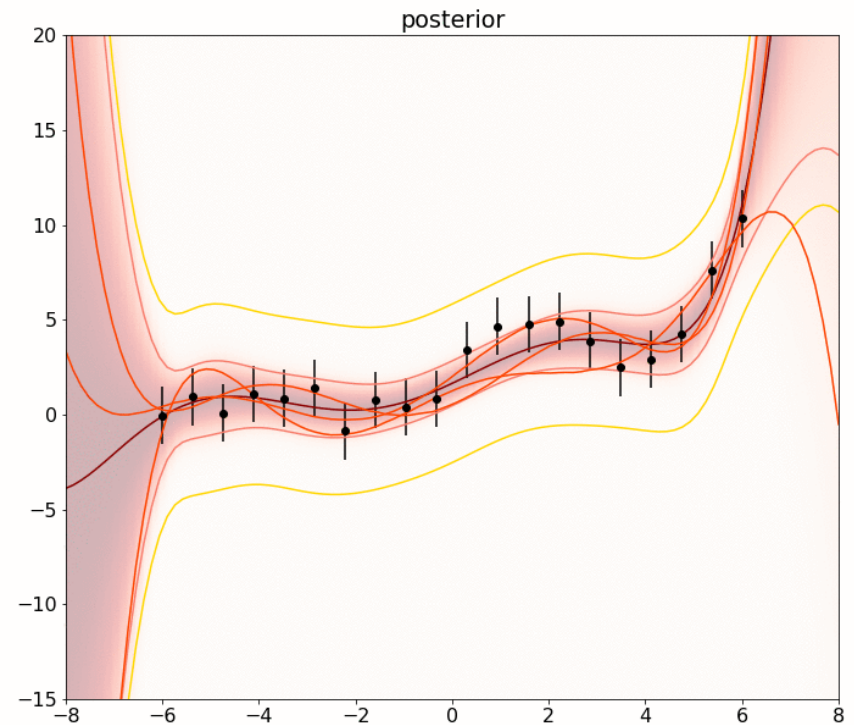
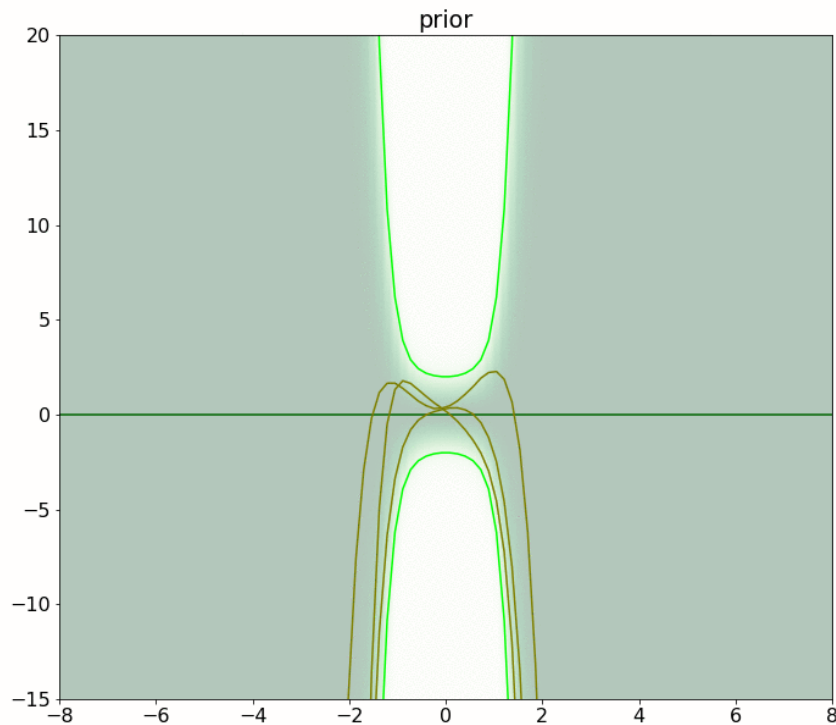
Polynomial features

- $\phi(x) = [1, x, x^2, x^3]$



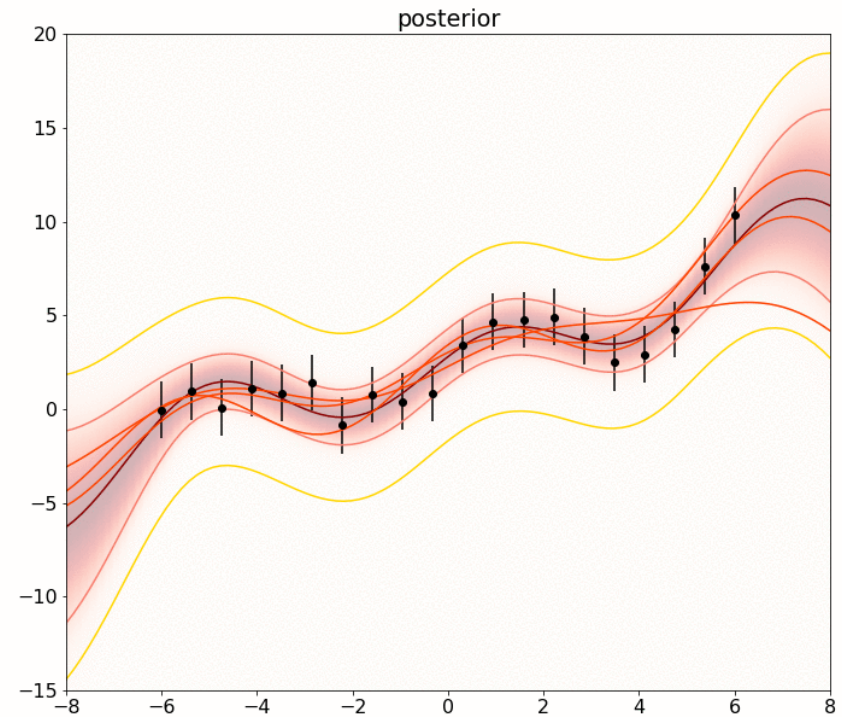
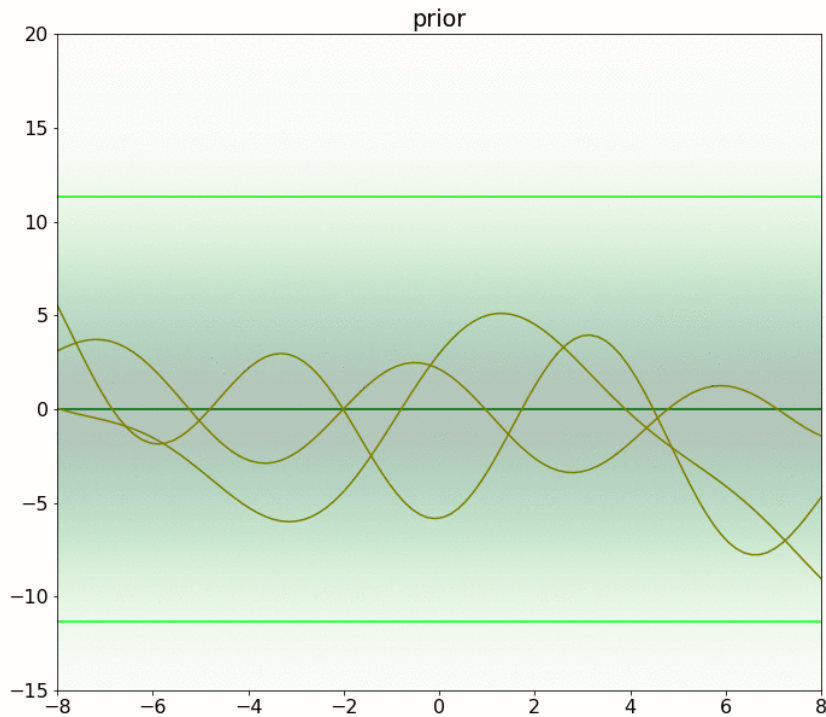
Polynomial features

■ $\phi(x) = [1, x, x^2, \dots, x^6]$



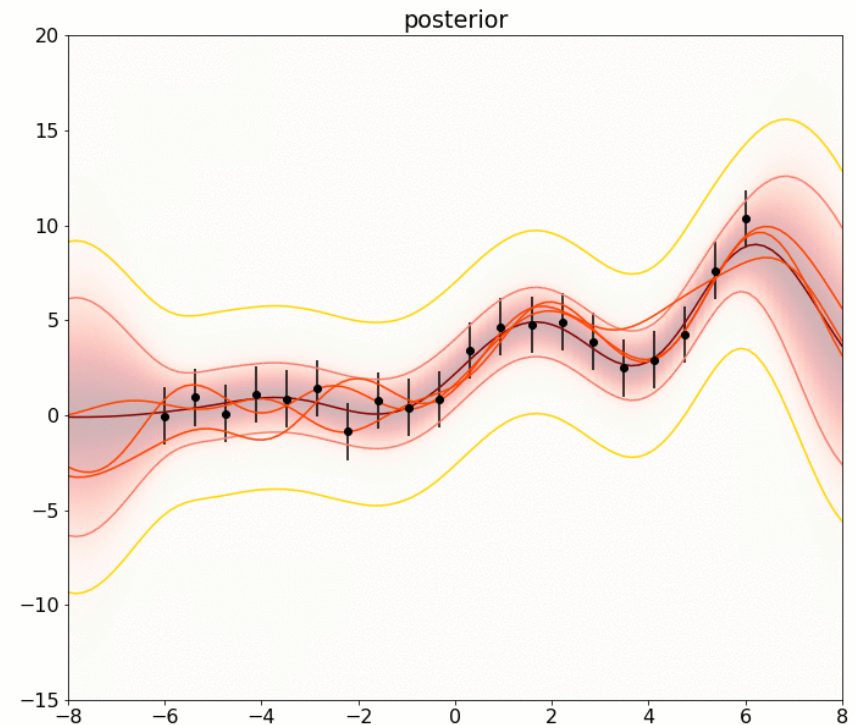
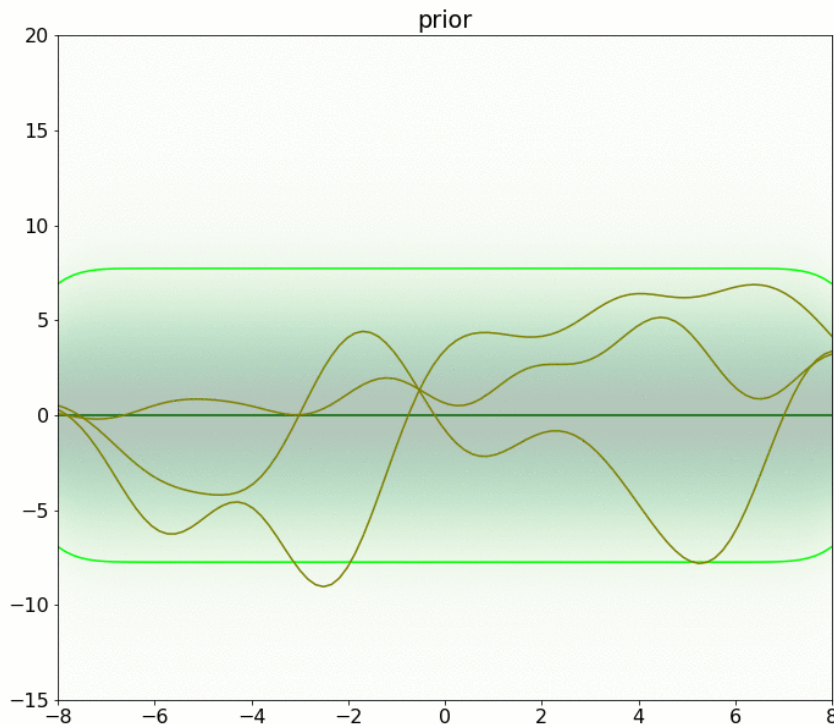
Fourier features

■ $\phi(x) = [\sin(x), \sin(2x), \dots, \cos(x), \cos(2x), \dots]$



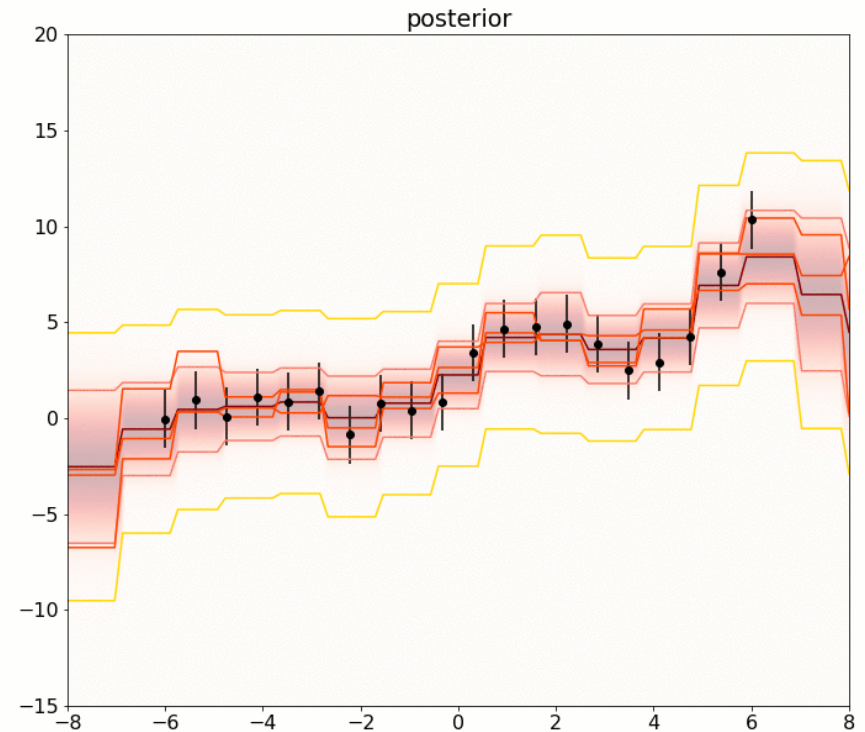
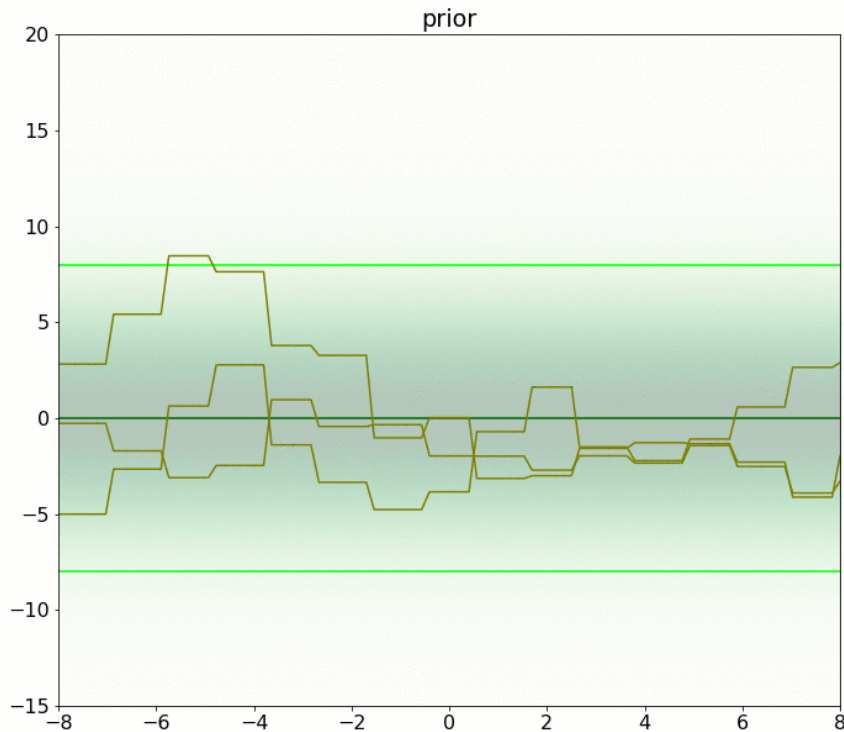
Bell features

- $\phi(x) = [e^{-\frac{1}{2}(x-8)}, e^{-\frac{1}{2}(x-7)}, \dots]$



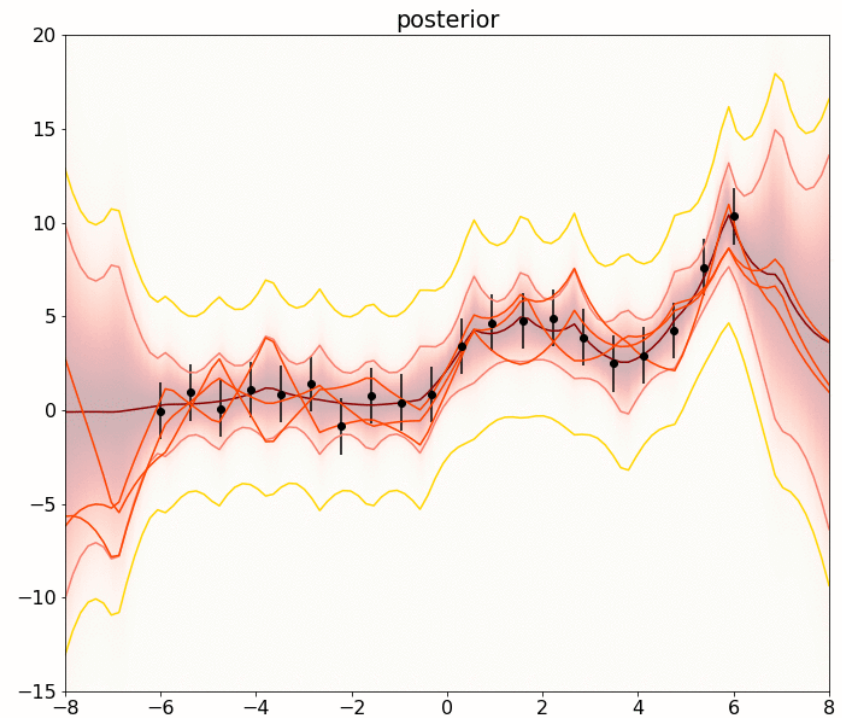
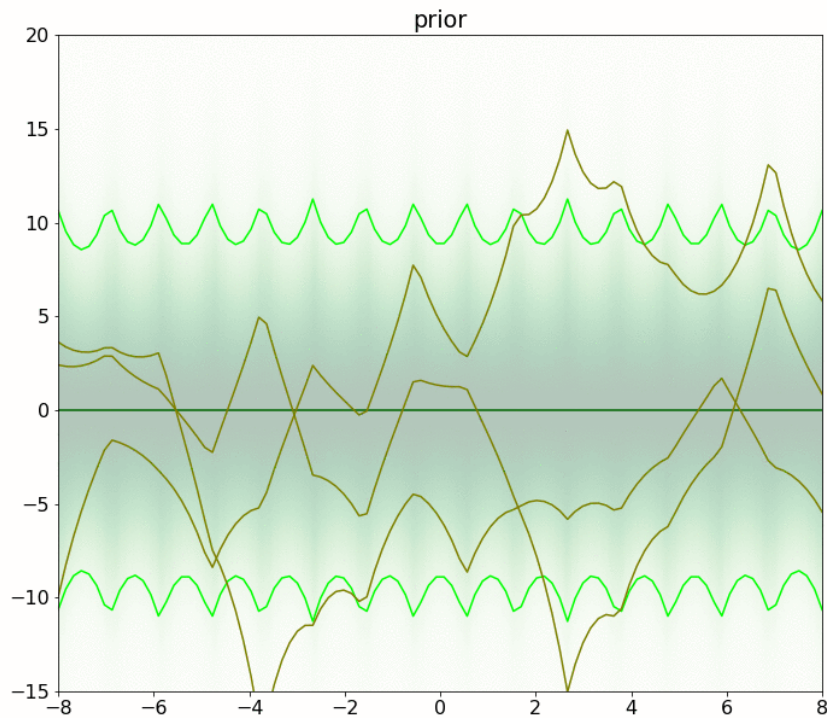
Heaviside features

- $\phi(x) = -1 + 2[H(x - 8), H(x - 7), \dots]$



Exponential features

■ $\phi(x) = [e^{-|x-8|}, e^{-|x-7|}, \dots]$



Can we do better?

- Example: learn the best features

$$\phi(x) = [e^{-|x-\theta_1|/\theta_0}, e^{-|x-\theta_2|/\theta_0}, \dots]$$

- The parameters $\Theta = \{\theta_0, \dots, \theta_n\}$ are called **hyperparameters**.
- Hyperparameters are nonlinear.
 - No closed-form. No Gaussian...
- We'll get back to it latter.

Alternative: higher number of features

- Can we use infinite features? Using the **kernel trick**!
- The predictive posterior can be written as:

$$\begin{aligned}\hat{f}_* &= \phi_*^T \Sigma \phi_X (\phi_X^T \Sigma \phi_X + \sigma^2 I)^{-1} \mathbf{y} \\ \sigma_{f_*}^2 &= \phi_*^T \Sigma \phi_* - \phi_*^T \Sigma \phi_X (\phi_X^T \Sigma \phi_X + \sigma^2 I)^{-1} \phi_X^T \Sigma \phi_*\end{aligned}$$

- All the operations with features are dot products. The result is always a scalar.
- Kernel function: $k(x_1, x_2) = \phi(x_1)^T \Sigma \phi(x_2)$
 - This type of function is called a Mercer kernel.

Gaussian processes

- Rewrite the predictive posterior:

$$\begin{aligned}\hat{f}_* &= k(x_*, X) (k(X, X) + \sigma^2 I)^{-1} \mathbf{y} \\ \sigma_{f_*}^2 &= k(x_*, x_*) - k(x_*, X) (k(X, X) + \sigma^2 I)^{-1} k(X, x_*)\end{aligned}$$

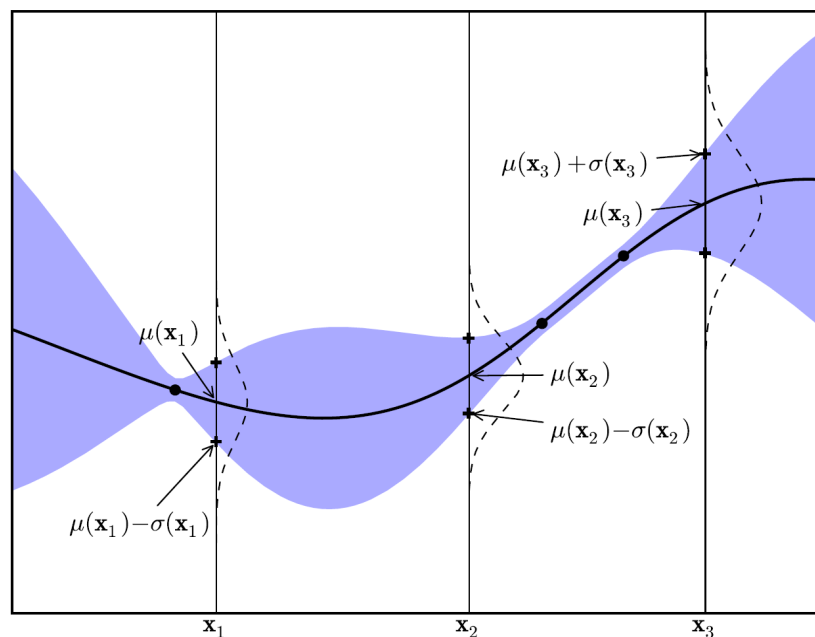
- If we use a Mercer kernel, we have a covariance matrix, a covariance vector and a variance term.
 - $k(x_1, x_2)$ measures the correlation/similarity of x_1 and x_2 .
- The predictive posterior is a conditional Gaussian of:

$$\begin{pmatrix} \mathbf{y} \\ f_* \end{pmatrix} \sim N \left(0, \begin{bmatrix} k(X, X) + \sigma^2 I & k(X, x_*) \\ k(x_*, X) & k(x_*, x_*) \end{bmatrix} \right)$$

Gaussian processes

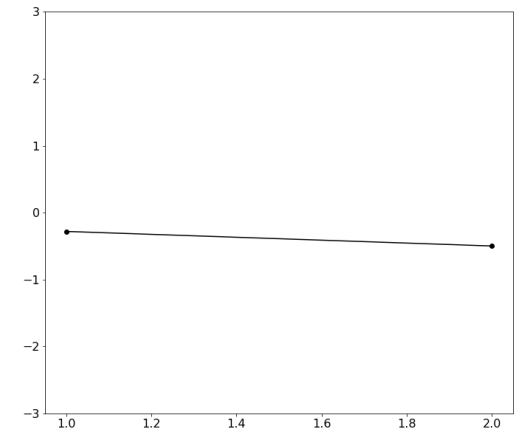
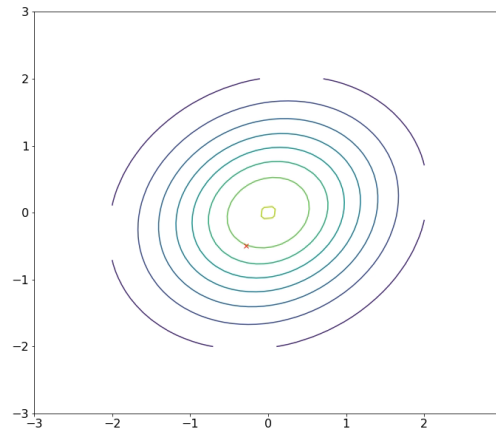
Definition: A Gaussian process is a collection of random variables, any finite of which have a joint Gaussian distribution.

$$\begin{pmatrix} \mathbf{y} \\ f_* \end{pmatrix} \sim N\left(0, \begin{bmatrix} k(X, X) + \sigma^2 I & k(X, x_*) \\ k(x_*, X) & k(x_*, x_*) \end{bmatrix}\right)$$

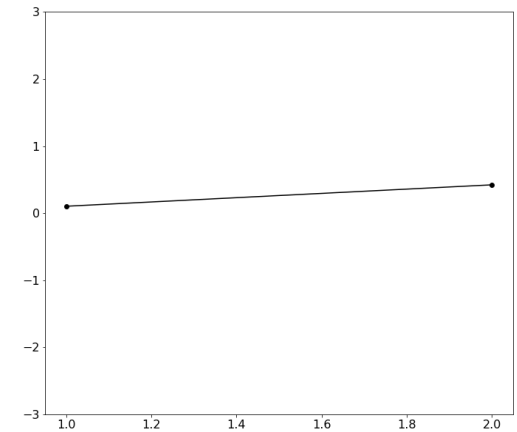
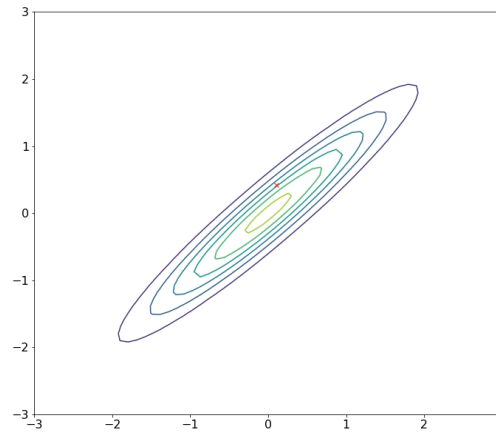


GPs as joint distributions

Small correlation

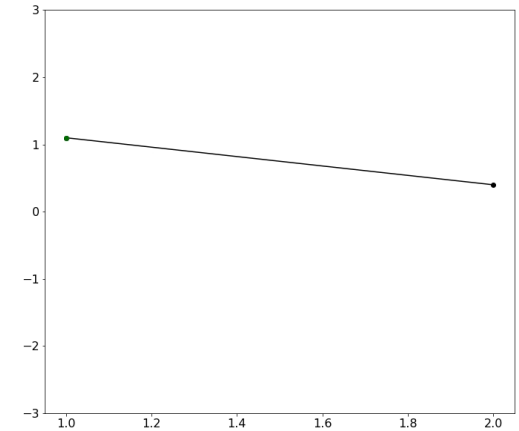
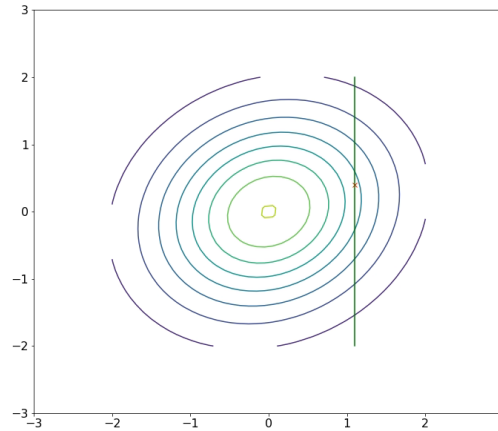


Large correlation

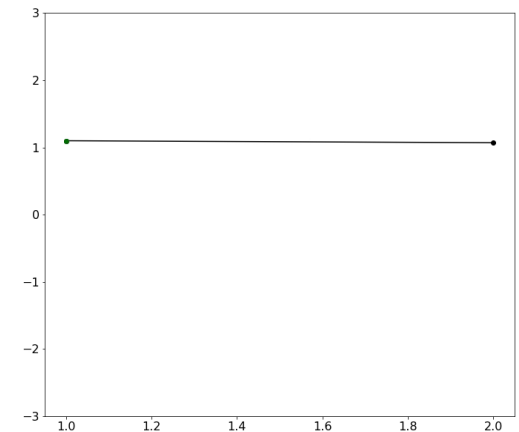
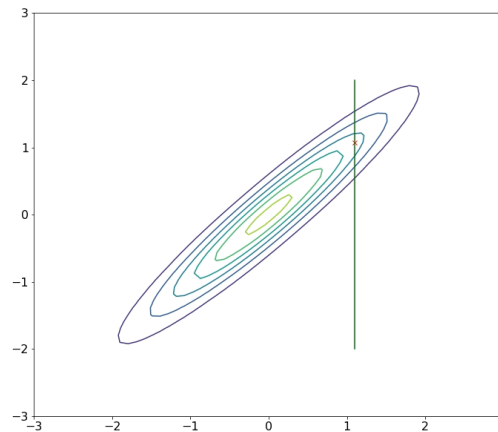


GPs as joint distributions

Small correlation

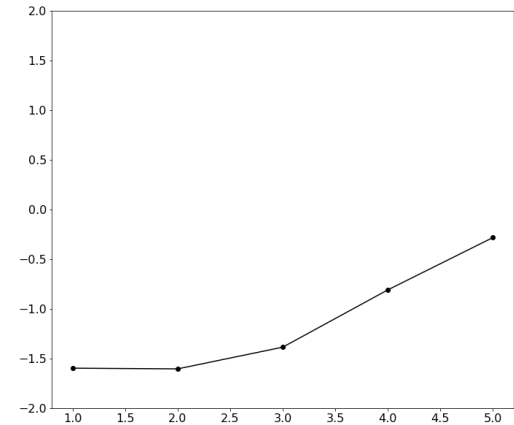
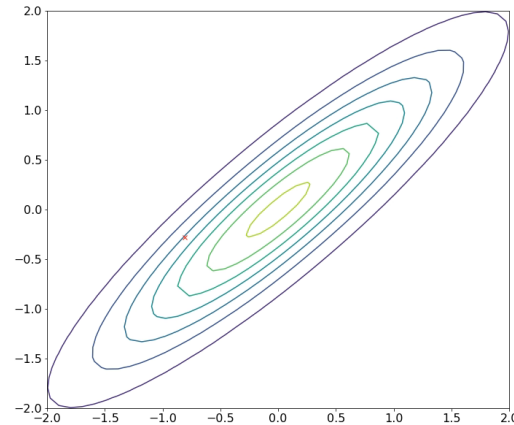


Large correlation



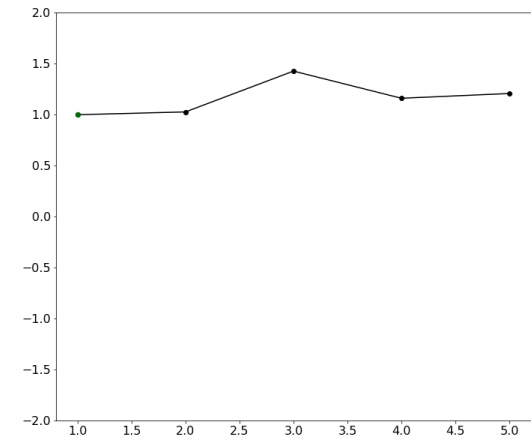
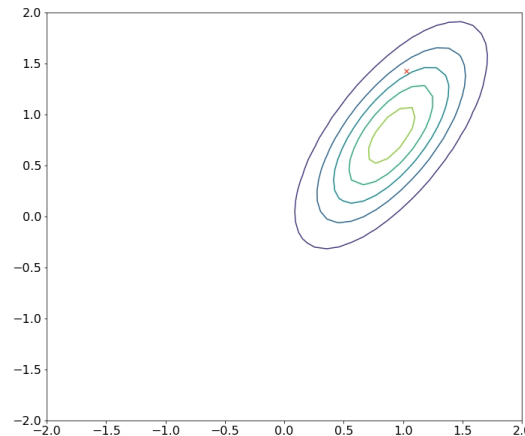
GPs as joint distributions

Joint



$$\Sigma = \begin{pmatrix} 1 & 0,9 & 0,8 & 0,6 & 0,4 \\ 0,9 & 1 & 0,9 & 0,8 & 0,6 \\ 0,8 & 0,9 & 1 & 0,9 & 0,8 \\ 0,6 & 0,8 & 0,9 & 1 & 0,9 \\ 0,4 & 0,6 & 0,8 & 0,9 & 1 \end{pmatrix}$$
$$\mu = (0 \ 0)^T$$

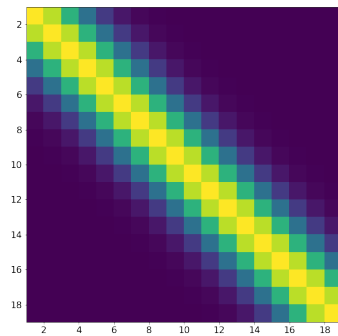
Conditional



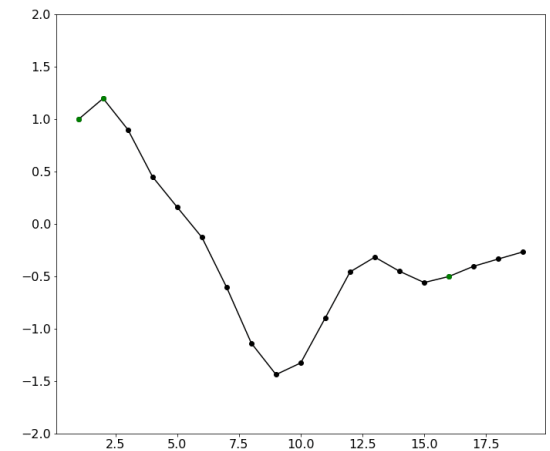
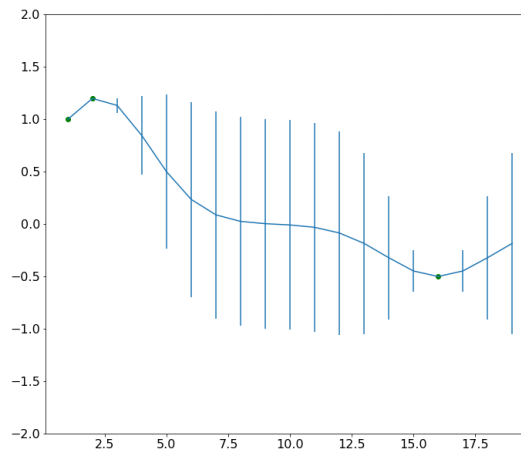
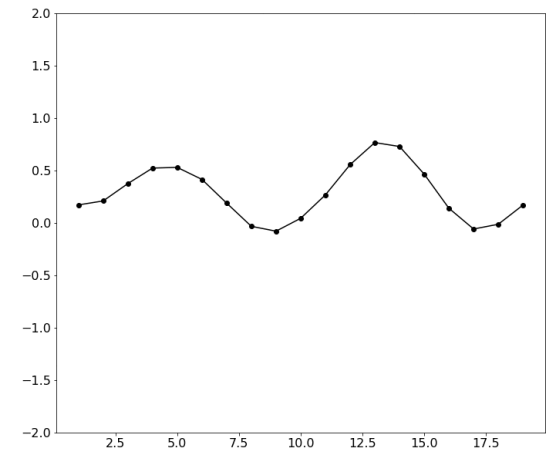
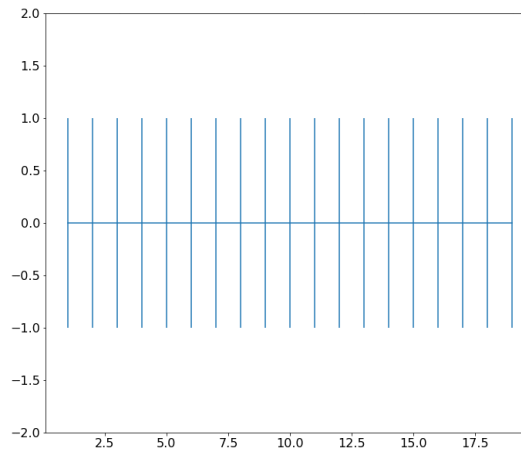
GPs as joint distributions

Joint

$\Sigma =$



Conditional



Nonparametric models

- Where are the model parameters/weights?

$$\begin{pmatrix} \mathbf{y} \\ f_* \end{pmatrix} \sim \mathcal{N} \left(0, \begin{bmatrix} \textcolor{red}{k(X, X)} + \sigma^2 I & \textcolor{green}{k(X, x_*)} \\ \textcolor{green}{k(x_*, X)} & \textcolor{blue}{k(x_*, x_*)} \end{bmatrix} \right)$$

- GPs are nonparametric models:
 - **Bad news:** (Computational) complexity increase with the number of data points.
 - **Good news:** (Model) complexity increase with the number of data points.

- GPs are distributions over functions:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

$$m(x) = \mathbb{E}[f(x)]$$

$$k(x, x') = \mathbb{E} \left[(f(x) - m(x))(f(x') - m(x'))^T \right]$$

Kernels

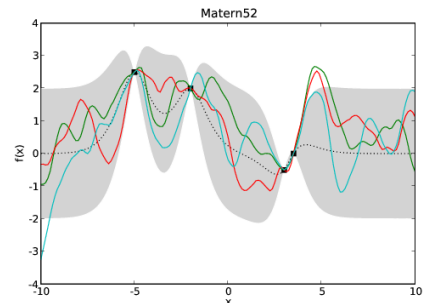
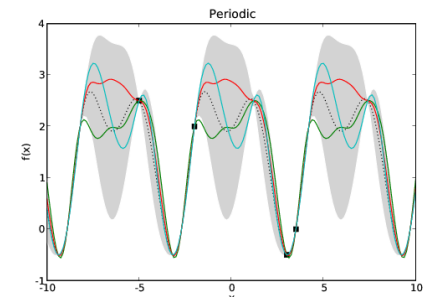
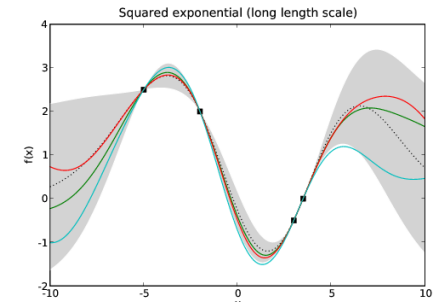
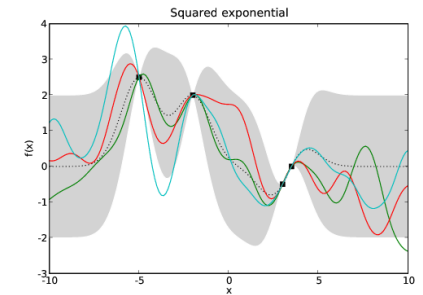
■ Commonly-used kernels (Rasmussen and Williams 2006):

- Radial Basis Functions (RBFs): $r(x, x') = |x - x'|$

	covariance function	expression
RBFs {	constant	σ_0^2
	linear	$\sum_{d=1}^D \sigma_d^2 x_d x'_d$
	polynomial	$(\mathbf{x} \cdot \mathbf{x}' + \sigma_0^2)^p$
	squared exponential	$\exp(-\frac{r^2}{2\ell^2})$
	Matérn	$\frac{1}{2^{\nu-1}\Gamma(\nu)} \left(\frac{\sqrt{2\nu}}{\ell} r\right)^\nu K_\nu\left(\frac{\sqrt{2\nu}}{\ell} r\right)$
	exponential	$\exp(-\frac{r}{\ell})$
	γ -exponential	$\exp\left(-\left(\frac{r}{\ell}\right)^\gamma\right)$
	rational quadratic	$(1 + \frac{r^2}{2\alpha\ell^2})^{-\alpha}$
	neural network	$\sin^{-1}\left(\frac{2\tilde{\mathbf{x}}^\top \Sigma \tilde{\mathbf{x}'}}{\sqrt{(1+2\tilde{\mathbf{x}}^\top \Sigma \tilde{\mathbf{x}})(1+2\tilde{\mathbf{x}'^\top \Sigma \tilde{\mathbf{x}'})}}}\right)$

■ Others: Wiener process, splines...

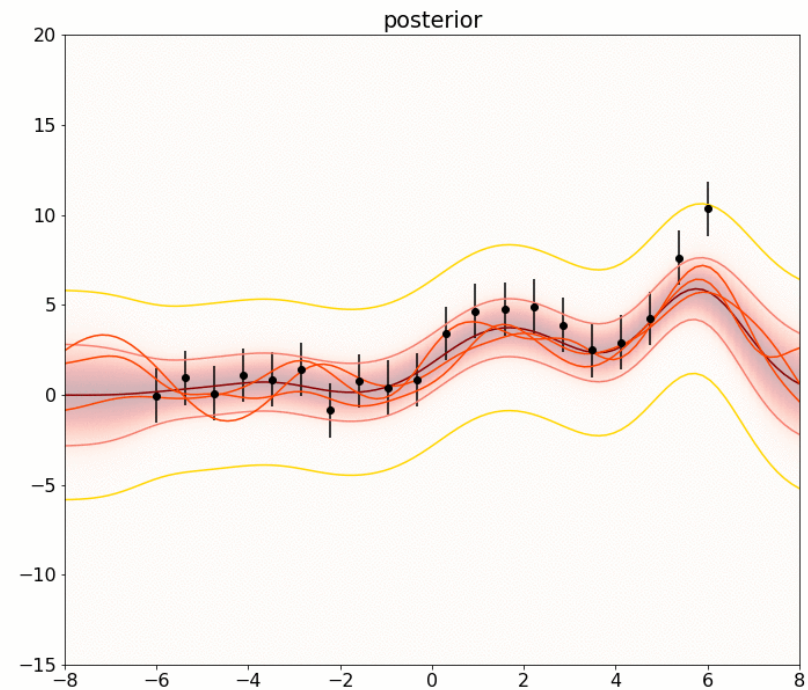
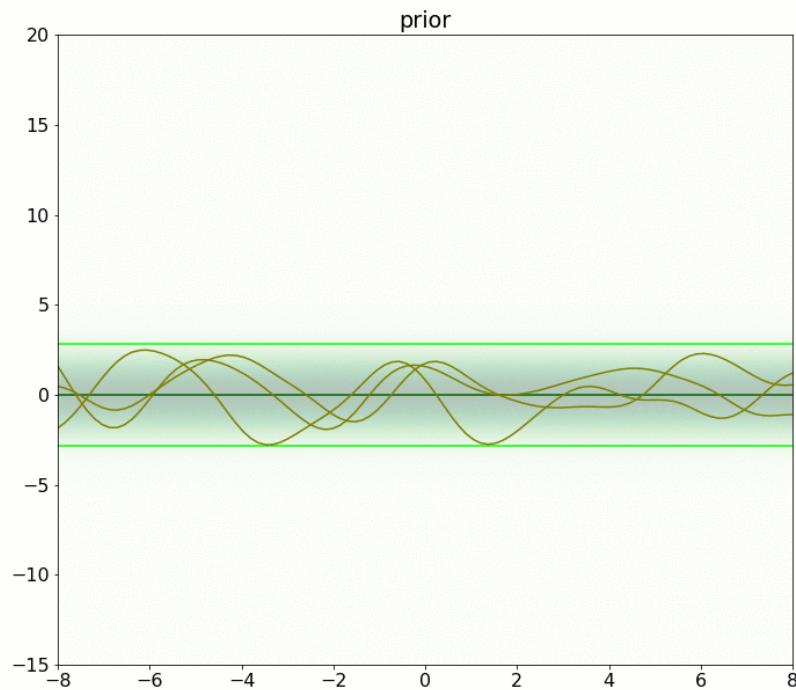
■ Not only real vectors: strings, paths...



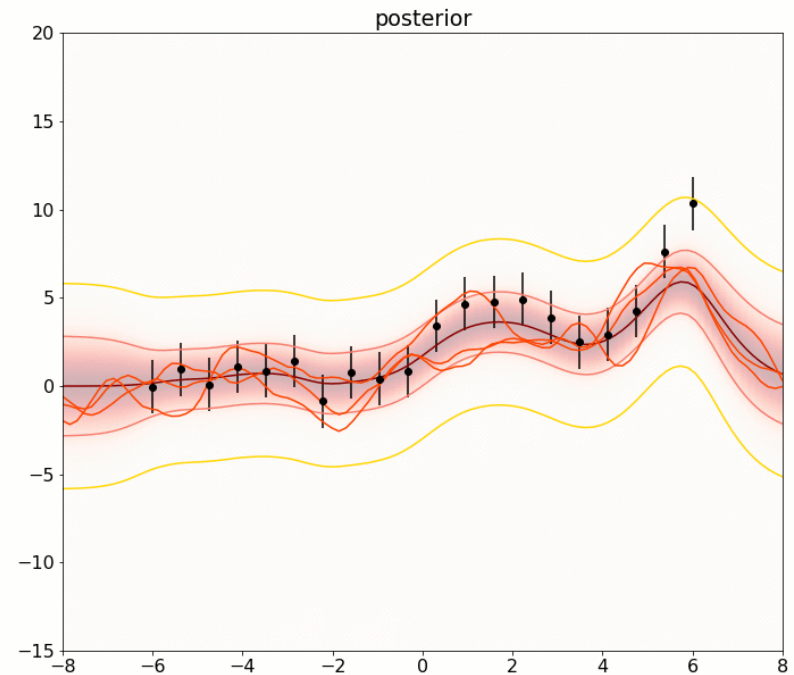
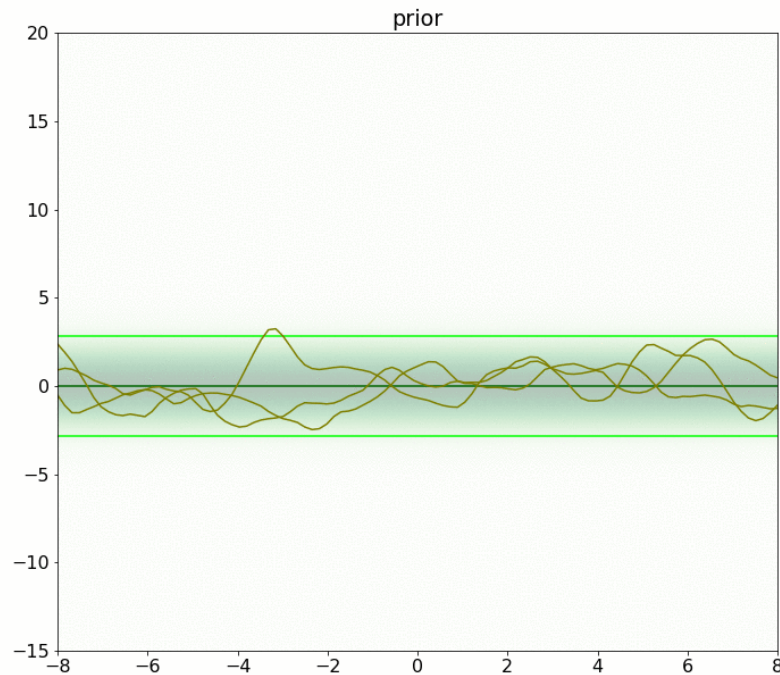
Squared exponential

GP . kern . RBF

$$k(x, x') = \exp\left(-\frac{|x - x'|^2}{2l^2}\right)$$



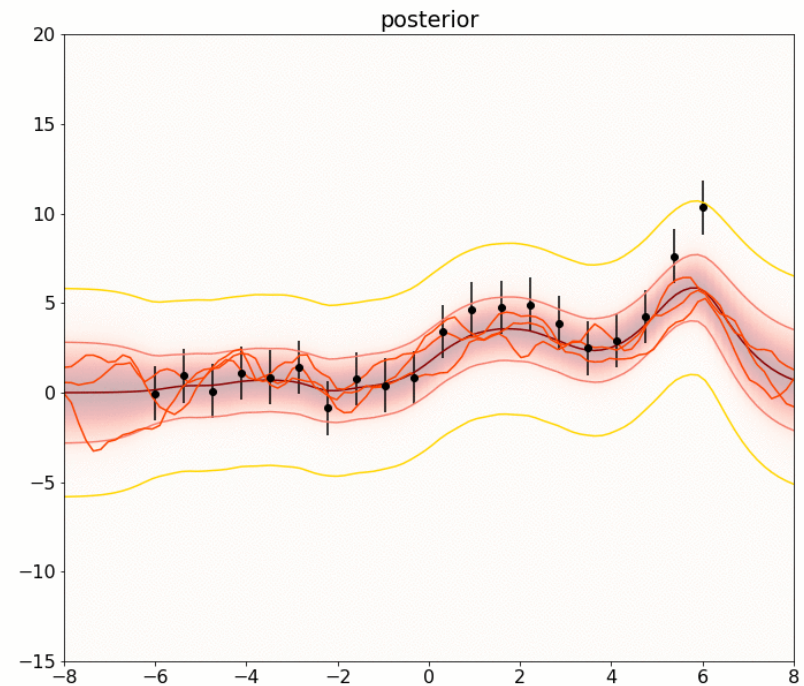
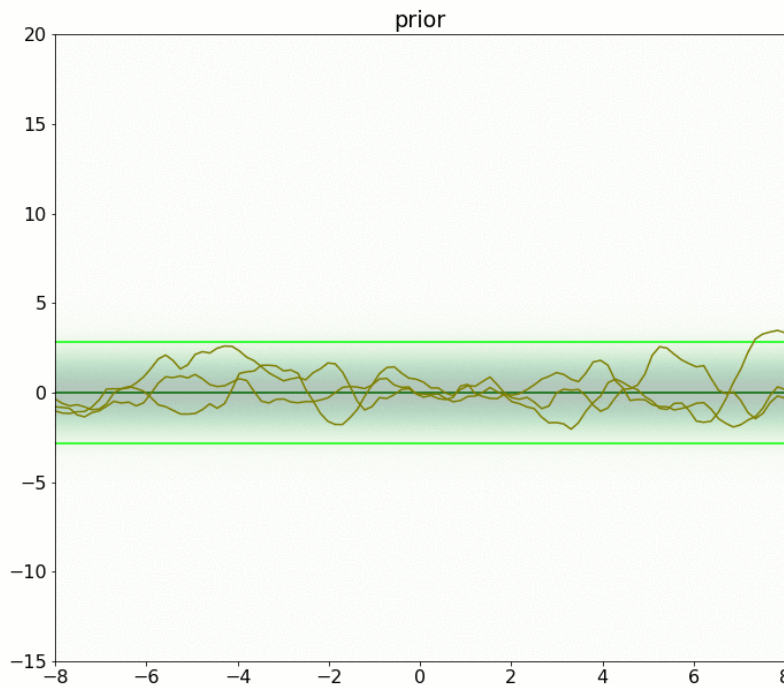
$$k(x, x') = \exp\left(-\sqrt{5} \frac{|x - x'|}{l}\right) \left(1 + \sqrt{5} \frac{|x - x'|}{l} + \frac{5}{3} \left(\frac{|x - x'|}{l}\right)^2\right)$$



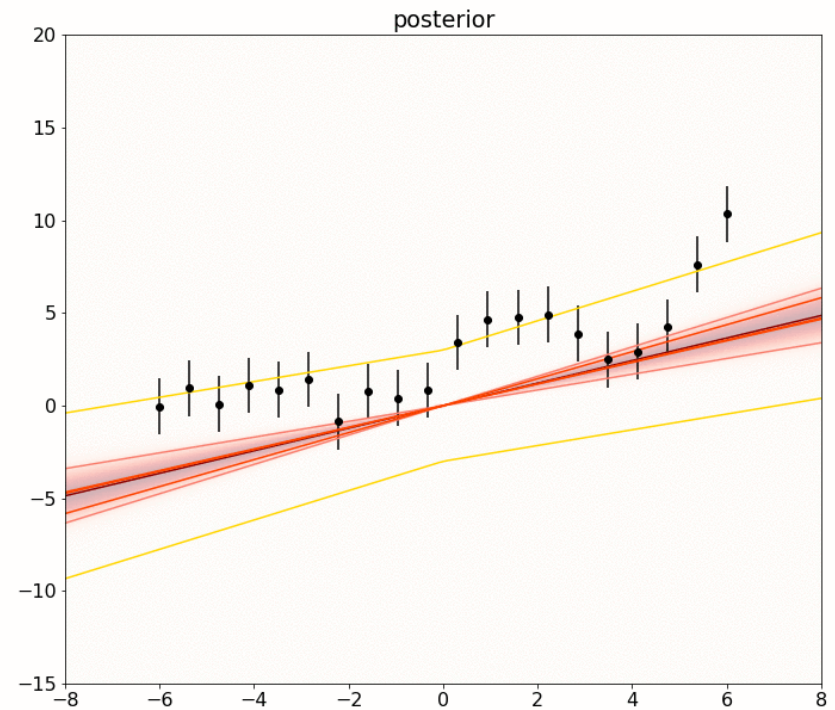
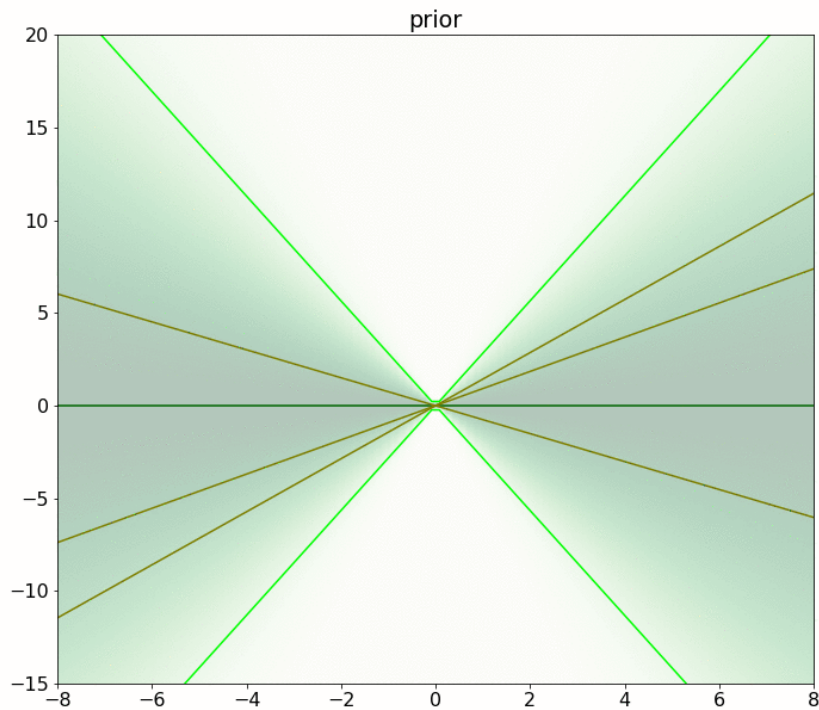
Matérn $\nu=3/2$

GPy.kern.Matern32

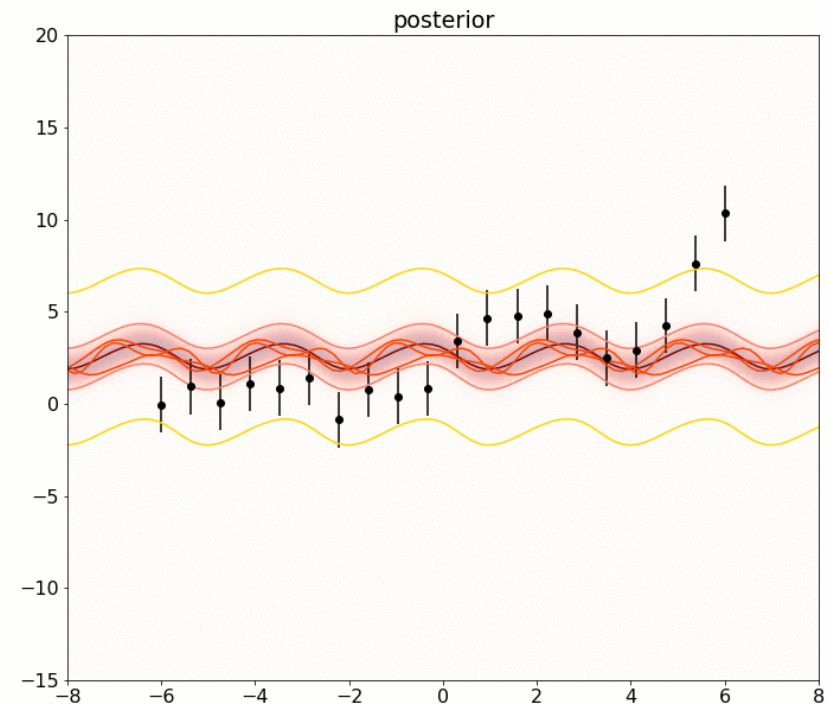
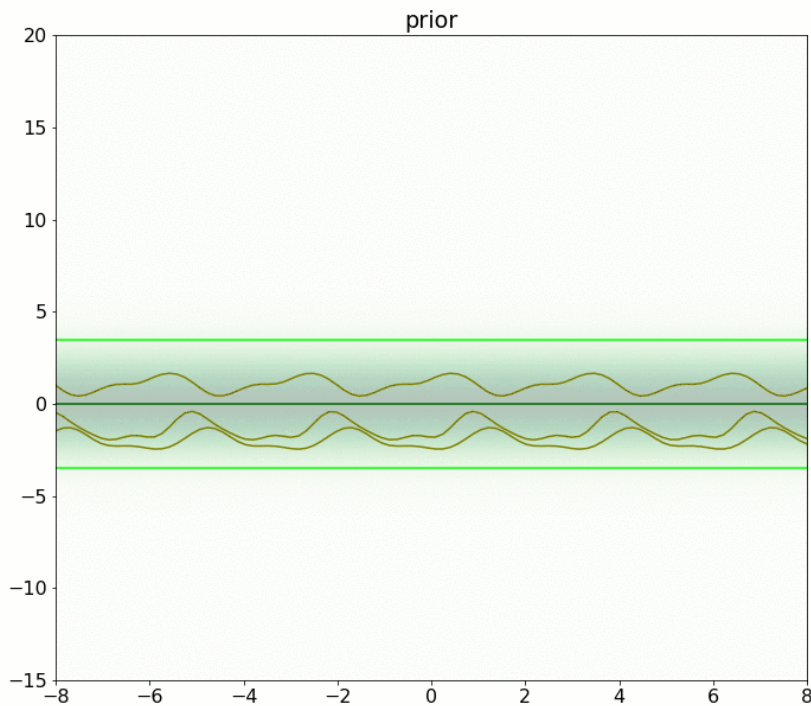
$$k(x, x') = \exp\left(-\sqrt{3}\frac{|x - x'|}{l}\right)\left(1 + \sqrt{3}\frac{|x - x'|}{l}\right)$$



$$k(x, x') = \sum_{d=1}^D \sigma_d^2 x_d x'_d$$



$$k(x, x') = \exp\left(-\frac{1}{2} \sum_{d=1}^D \left[\frac{\sin\left(\frac{\pi}{T_d}(x_d - x'_d)\right)}{l_d} \right]^2\right)$$



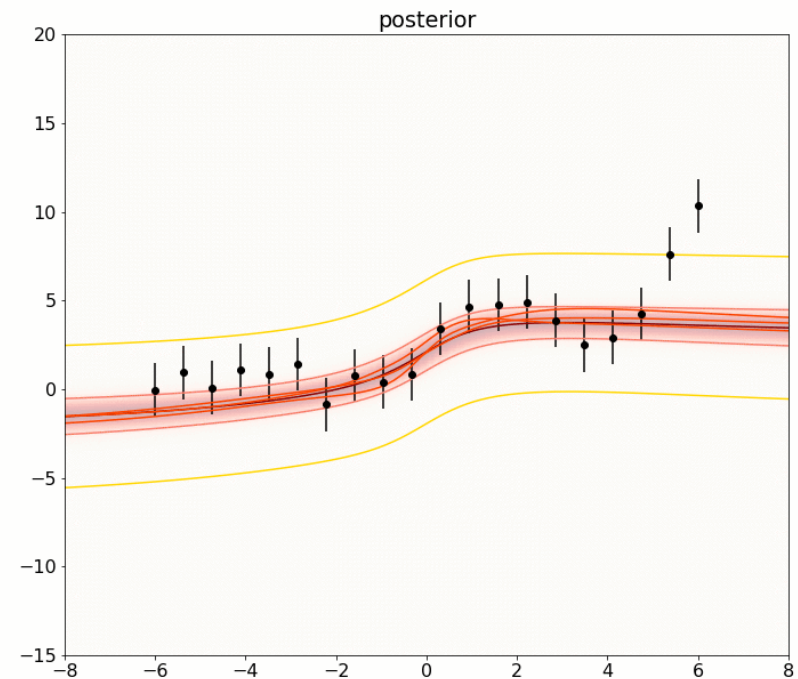
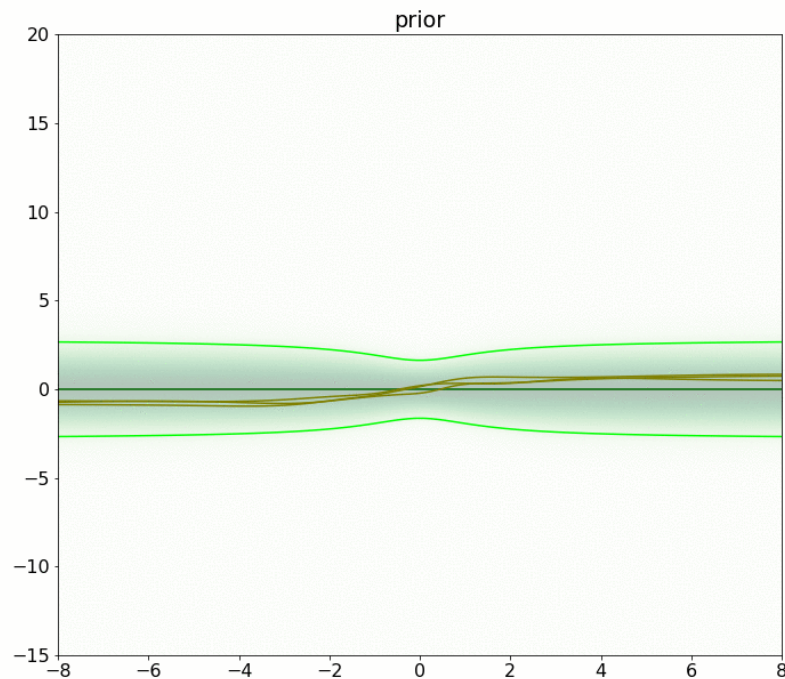
Neural network

(perceptron with erf activation)

GPY.kern.MLP

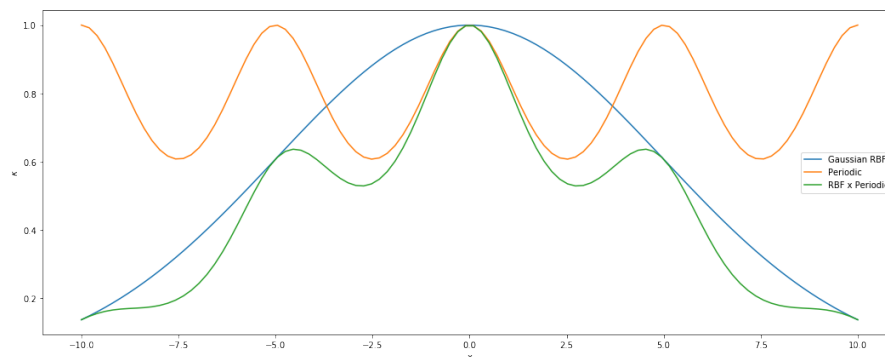
$$k(x, x') = \sin^{-1} \left(\frac{2\tilde{x}^T \Sigma \tilde{x}'}{\sqrt{(1+2\tilde{x}^T \Sigma \tilde{x})(1+2\tilde{x}'^T \Sigma \tilde{x}')}} \right)$$

$$\tilde{x} = (1, x_1, \dots, x_d)$$

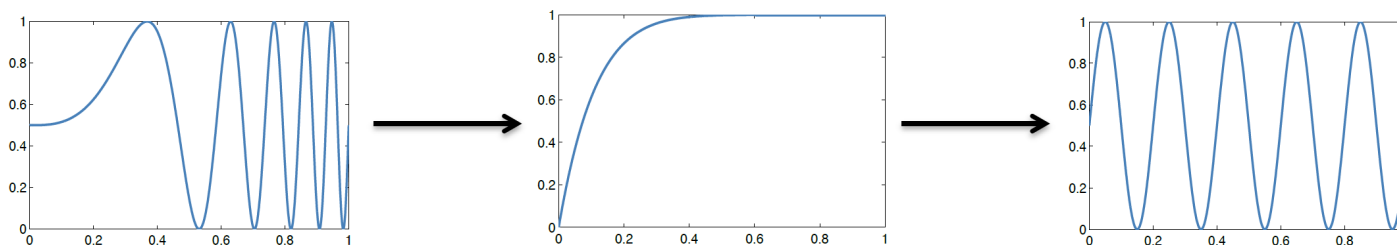


How many kernels?

- We can combine them!
 - Scaled kernel: $\alpha \cdot k(x, x')$
 - Sum of kernels: $k_1(x, x') + k_2(x, x')$
 - Product of kernels: $k_1(x, x') \cdot k_2(x, x')$



- Mapped inputs: $k(\psi(x), \psi(x'))$

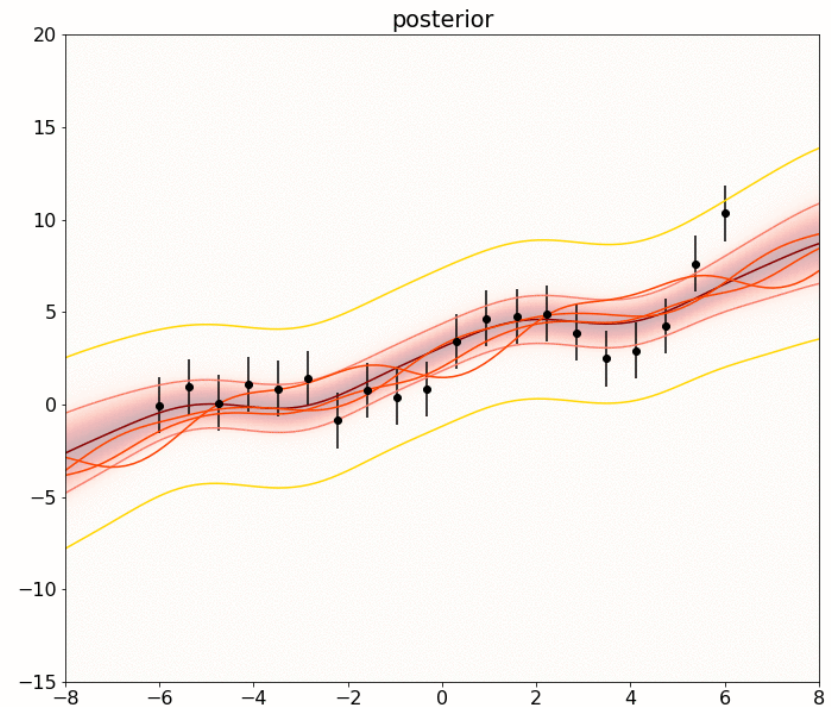
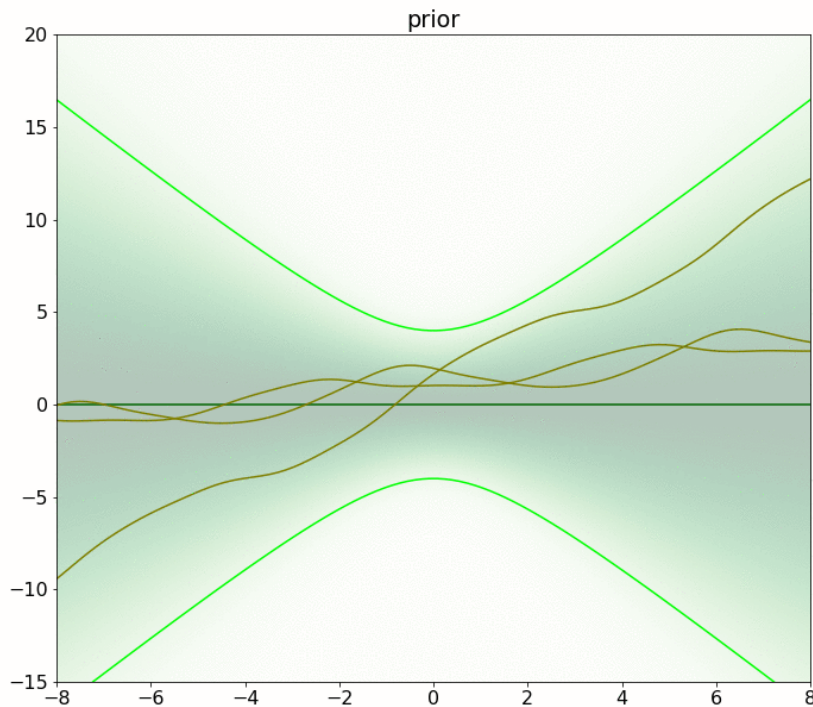


Linear + periodical

GPy.kern.Add

...with adjusted period

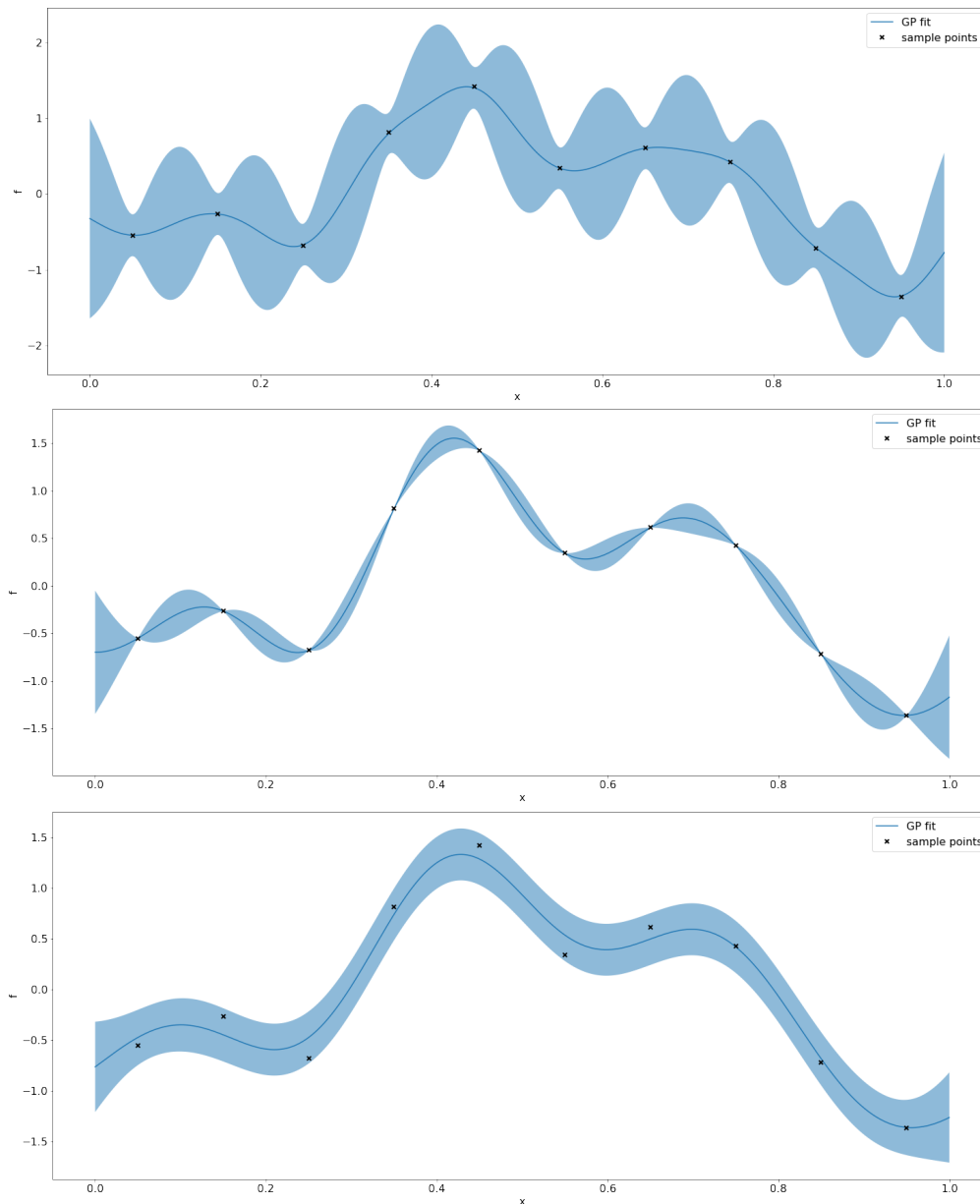
$$k(x, x') = \sum_{d=1}^D \sigma_d^2 x_d x'_d + \exp \left(-\frac{1}{2} \sum_{d=1}^D \left[\frac{\sin\left(\frac{\pi}{T_d}(x_d - x'_d)\right)}{l_d} \right]^2 \right)$$



Kernel hyperparameters

- A kernel might have multiple hyperparameters
 - Length-scale
 - Signal variance
 - Signal noise...

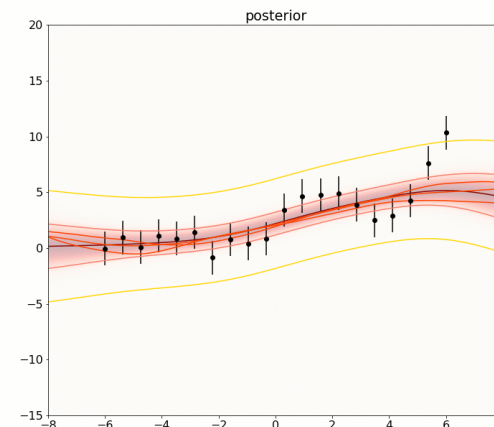
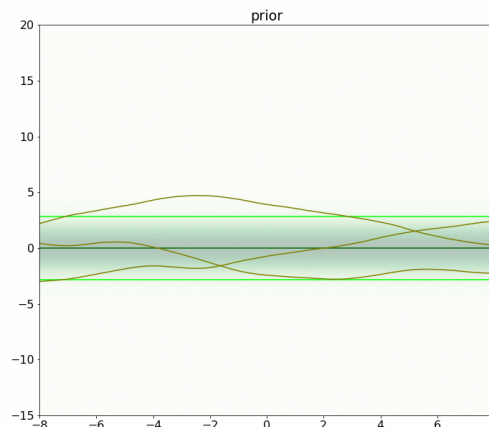
$$k(x, x') = \sigma_s e\left(-\frac{|x-x'|^2}{2l^2}\right) + \mathbb{I}_{x=x'}\sigma_n$$



Matern 5/2

...with different lengthscale

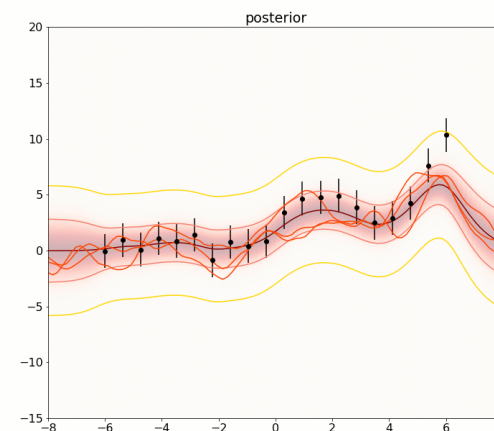
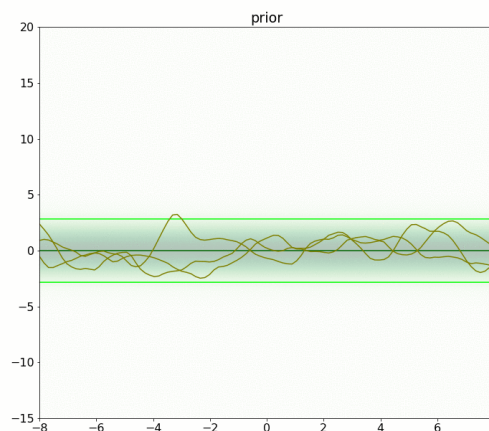
$l=5$



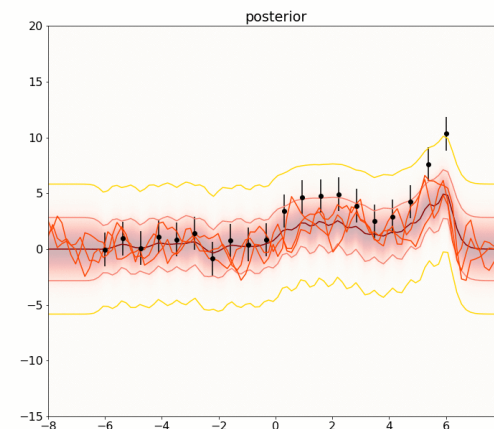
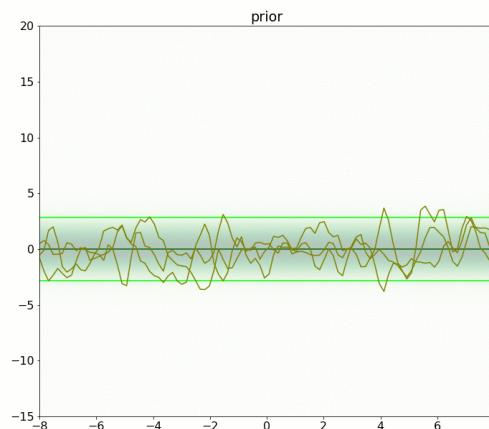
$$k(x, x') = \exp\left(-\sqrt{5}\frac{r}{l}\right)\left(1 + \sqrt{5}\frac{r}{l} + \frac{5}{3}\left(\frac{r}{l}\right)^2\right)$$

$$r = |x - x'|$$

$l=1$



$l=0.3$



Automatic Relevance Determination

- Used in multidimensional RBFs:
 - Matern, square exponential...
 - Ignore unnecessary dimensions (like PCA)
- Each dimension has a different lengthscale

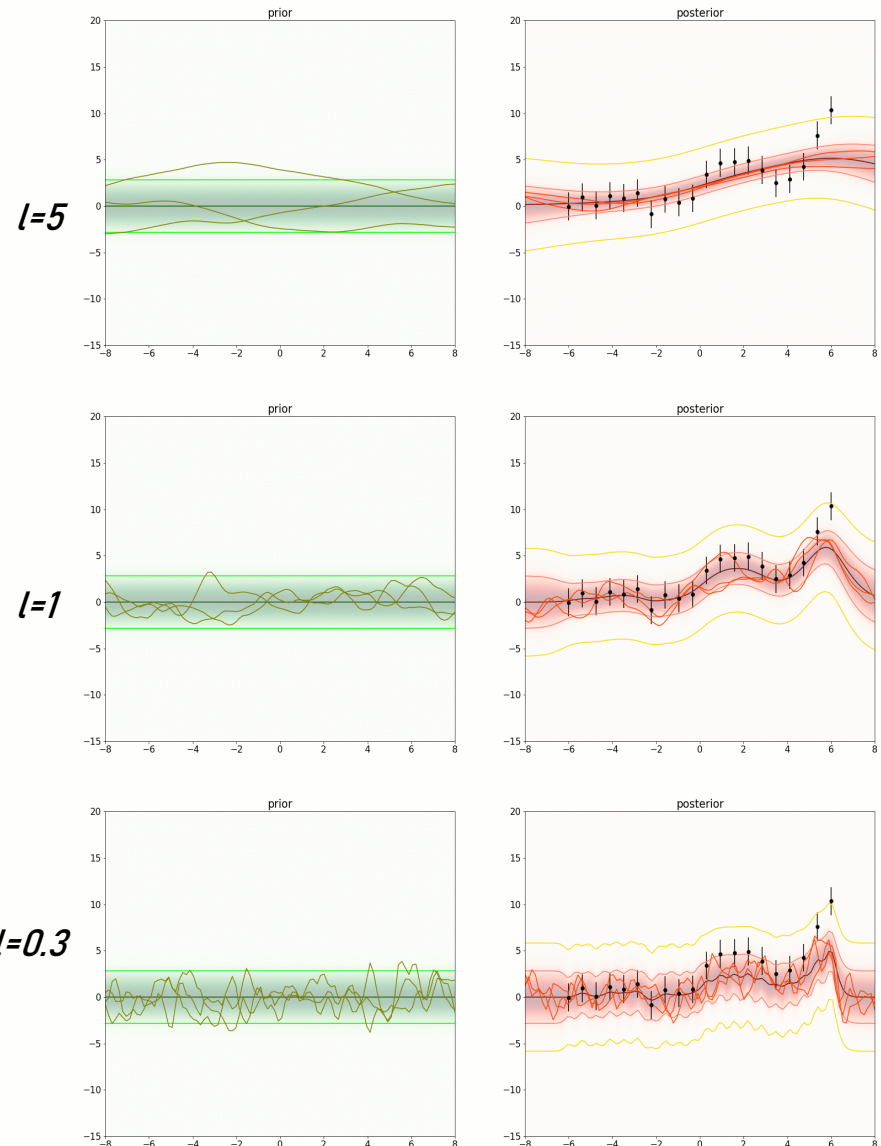
$$r = (x - x')^T \mathbf{\Lambda} (x - x')$$

$$\mathbf{\Lambda} = \text{diag}(1/l_1, \dots, 1/l_n)$$

Lengthscale $\uparrow \Rightarrow$ Correlation $\uparrow$

Correlation $\uparrow \Rightarrow$ Relevance $\downarrow$

- Intuition: When the correlation is high, knowing few values gives information about everything. You don't need more data in that dimension.



Learning hyperparameters

- Kernel hyperparameters are nonlinear.
 - No closed-form. No Gaussian...
- We can approximate the solution!
 - Empirical Bayes (type-II ML):
 - **MLE (or MAP) for the kernel hyperparameters.**
 - Warning: Underestimate uncertainty.
 - Expensive.
 - Similar alternative: Cross-validation.
 - Sampling (e.g.: MCMC – next lesson):
 - *Full Bayesian*
 - Result in a mixture of GPs
 - Much, much more expensive



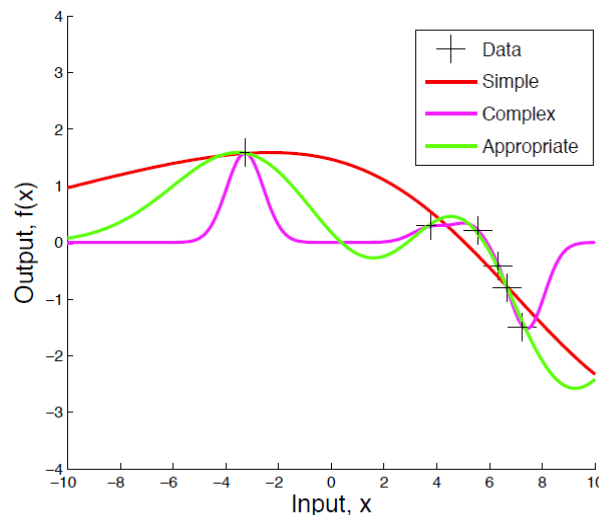
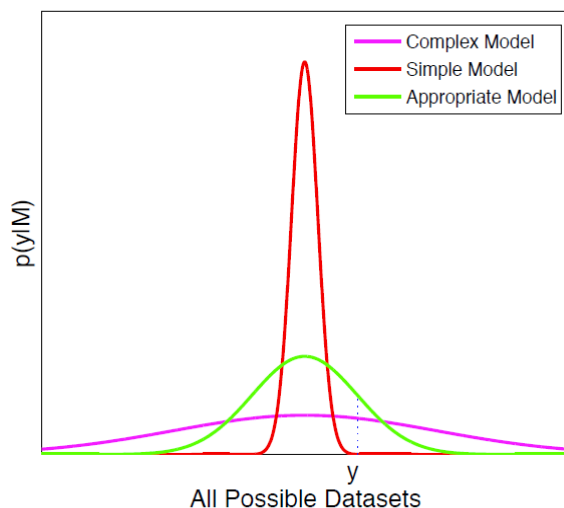
ML for kernel hyperparameters

- Maximize the marginal log-likelihood:

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^T K_{\theta}^{-1} \mathbf{y} - \frac{1}{2} \log |K_{\theta}| - \frac{n}{2} \log 2\pi$$

$$\theta^{ML} = \arg \max_{\theta} \log p(y \mid X, \theta)$$

- Automatically trade-off model complexity and model fit



Gaussian process regression

- Summary:

- Predict – Use predictive posterior:

$$\hat{f}_* = k(x_*, X) (k(X, X) + \sigma^2 I)^{-1} \mathbf{y}$$
$$\sigma_{f_*}^2 = k(x_*, x_*) - k(x_*, X) (k(X, X) + \sigma^2 I)^{-1} k(X, x_*)$$

- Learn – Maximize the marginal log-likelihood:

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^T \mathbf{K}_\theta^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_\theta| - \frac{n}{2} \log 2\pi$$

Colab (GP assignment):

<https://drive.google.com/file/d/1FRDhW8y5N3vBfaVm3Yub1ouhe5gZTDND/view?usp=sharing>

Gaussian process regression

- Algorithm 2.1 (Rasmussen and Williams):
 - Cholesky decomposition → Faster and numerically stable

$$\hat{f}_* = k_*^T \underbrace{K_X^{-1} \mathbf{y}}_{\alpha}$$

$$K_X = LL^T \Rightarrow K_X^{-1} = (L^T)^{-1}(L)^{-1}$$

$$\alpha = K_X^{-1} \mathbf{y} = (L^T)^{-1} L^{-1} \mathbf{y}$$

input: X (inputs), $\mathbf{y}$ (targets), k (covariance function), σ_n^2 (noise level),
 $\mathbf{x}_*$ (test input)

2: $L := \text{cholesky}(K + \sigma_n^2 I)$

$\alpha := L^\top \setminus (L \setminus \mathbf{y})$

4: $\bar{f}_* := \mathbf{k}_*^\top \alpha$

$\mathbf{v} := L \setminus \mathbf{k}_*$

6: $\mathbb{V}[f_*] := k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{v}^\top \mathbf{v}$

$\log p(\mathbf{y}|X) := -\frac{1}{2} \mathbf{y}^\top \alpha - \sum_i \log L_{ii} - \frac{n}{2} \log 2\pi$

8: **return:** $\bar{f}_*$ (mean), $\mathbb{V}[f_*]$ (variance), $\log p(\mathbf{y}|X)$ (log marginal likelihood)

} predictive mean eq. (2.25)

} predictive variance eq. (2.26)

eq. (2.30)

Non-zero mean and hierarchical Bayes

- Adding a fixed mean function just *shifts* the Gaussian process.

- Equivalent to computing the GP on the residuals

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')) = g(x) + h(x)$$

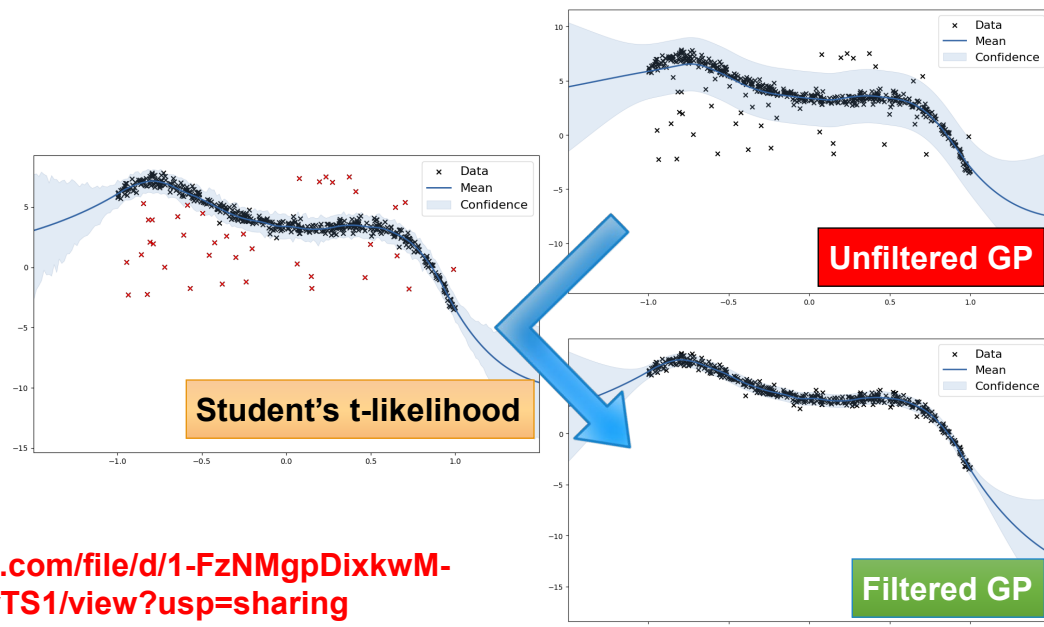
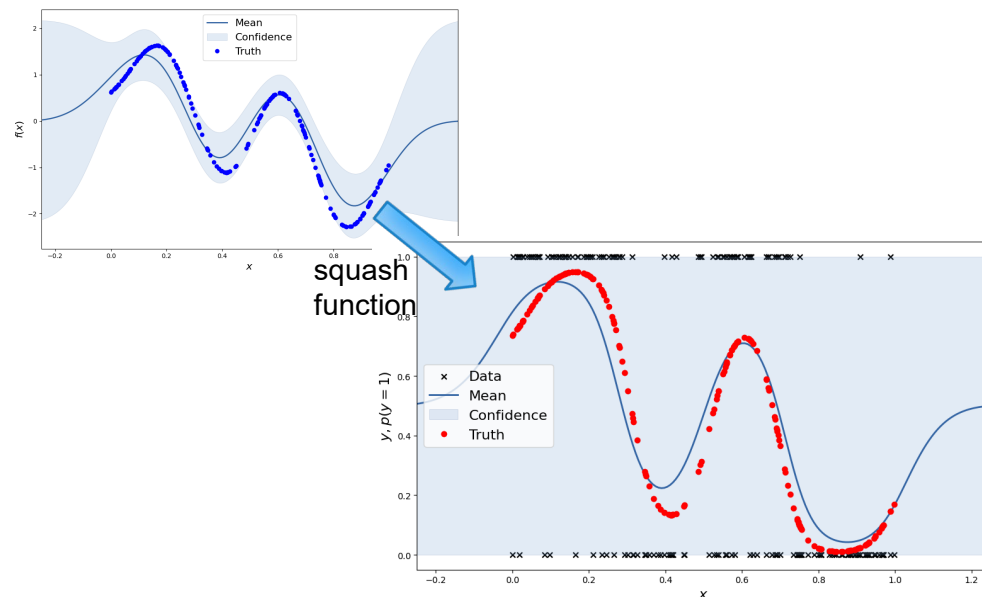
$$g(x) = \phi^T(x)w$$

$$h(x) \sim \mathcal{GP}(0, k(x, x'))$$

- Non-zero mean GP = parametric mean + zero mean GP
- We can combine GPs with Bayesian linear regression (it's the same!)
 - Sum of GPs is a GP
 - Example: Add a prior on $w \sim N(0, \Sigma)$
 - The posterior function $f(x)$ is still a GP. Analytical solution.
- We can also set a prior on the covariance $\sigma_s^2 \mathbf{K}(X, X)$:
 - We need a proper conjugate prior
 - $\sigma_s^2 \sim IG(\alpha, \beta)$ or $\mathbf{K} \sim Weibull(k, \lambda)$
 - Results in a Student's t process. Analytical solution!

Non-Gaussian likelihood

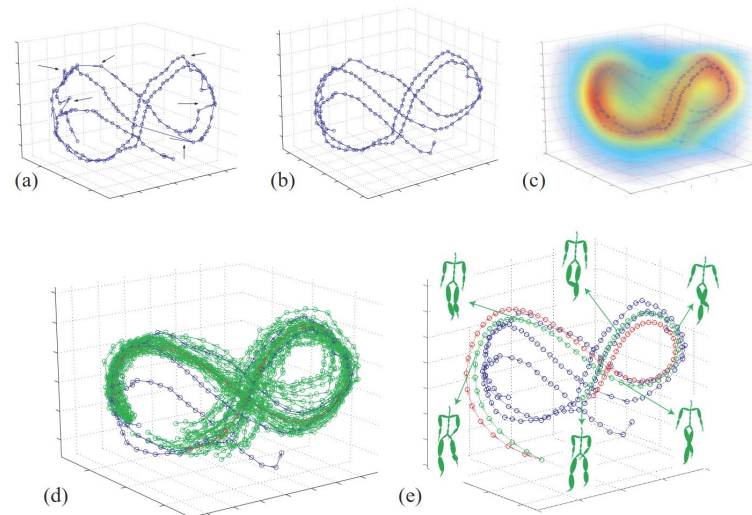
- No closed-form solution
 - Approximated inference (e.g.: Laplace, EP, MCMC, etc.)
- GPs for classification:
 - Concept: Logistic regression meets GPs
- Robust regression
 - Use long tail likelihoods (e.g.: Laplace, Student-t)
 - Use the 95% quantile for filtering



Beyond supervised learning

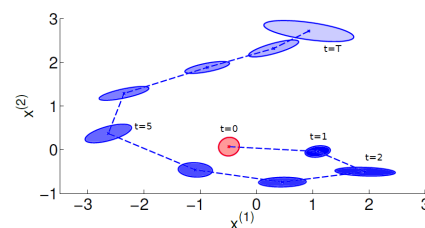
■ GPs for dimensionality reduction

- Gaussian process latent variable models (GPLVM)
- GP models mapping from latent variables Z to data points X
- GPLVM $\approx$ Kernel-PCA (nonlinear) + Probabilistic-PCA (Bayesian)

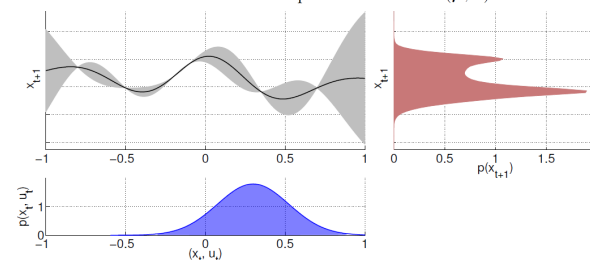


■ GPs for decision making

- GPs in reinforcement learning/bandits
- GPs for dynamic systems
- GPs for experimental design and optimization $\rightarrow$ Meta-learning



$$p(\mathbf{x}_{t+1}|\boldsymbol{\theta}) = \iiint \underbrace{p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)}_{\text{GP prediction}} \underbrace{p(\mathbf{x}_t, \mathbf{u}_t|\boldsymbol{\theta})}_{\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})} d\mathbf{f} d\mathbf{x}_t d\mathbf{u}_t$$



Advanced GPs

- Big data:
 - Sparse Gaussian processes
 - Inducing point methods...
- Deep GPs
 - Build a model made of layers of GPs
- GPs with deep kernels
 - Neural networks and GPs. Best of both!

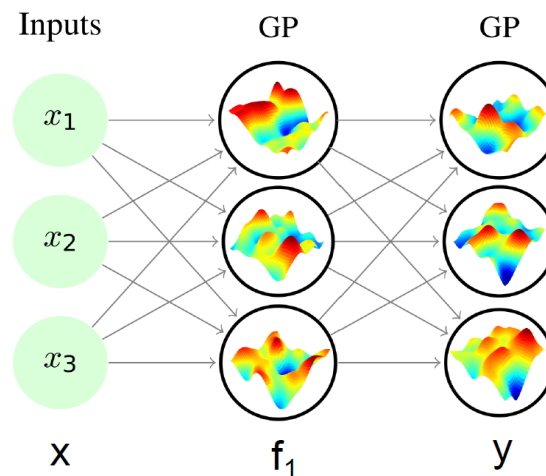


Image: M. Havasi et al.,

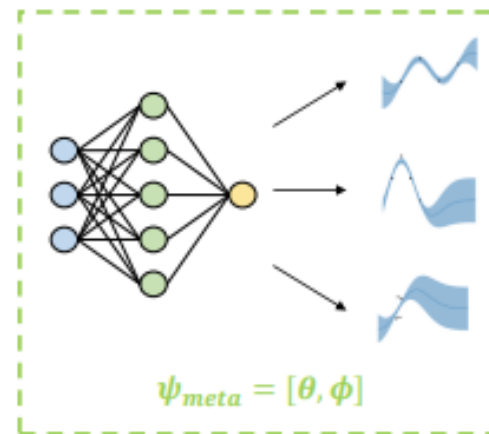
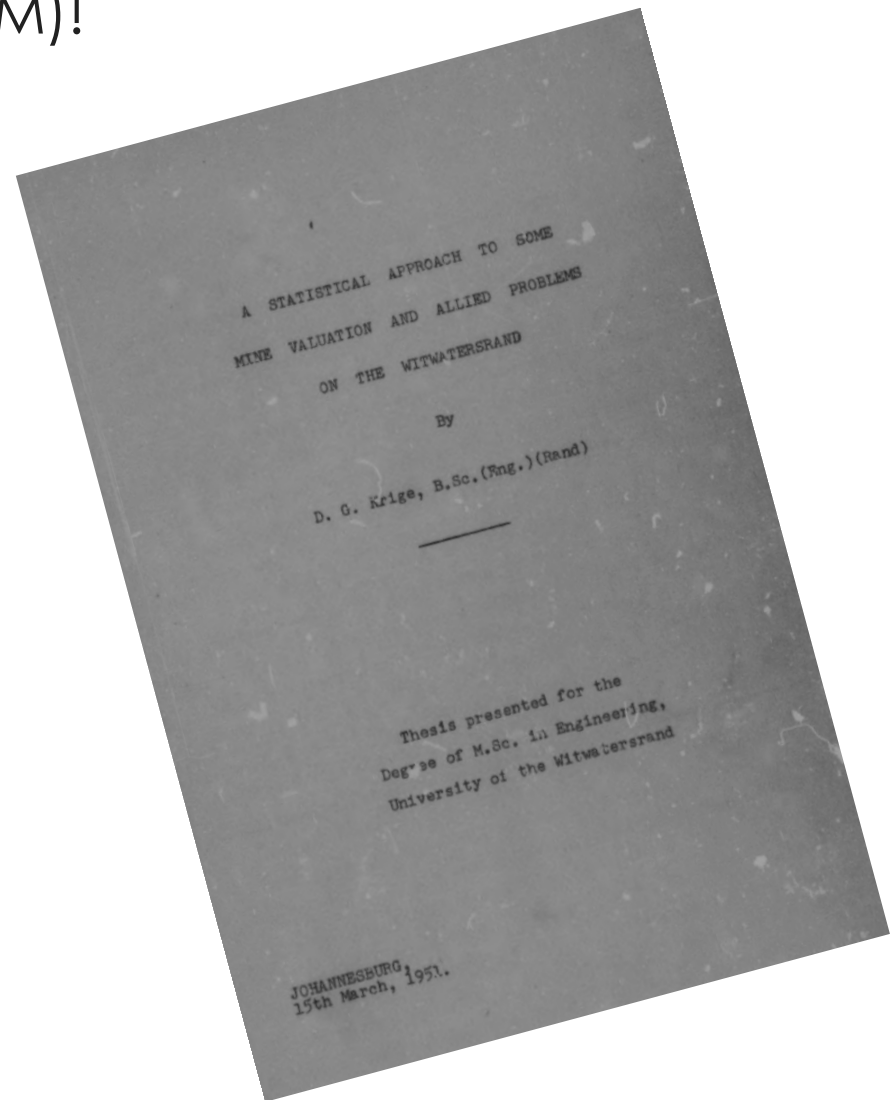


Image: JMHL

Gaussian processes (Kriging)

- It was a Master Thesis (TFM)!
 - From geology!
 - In Africa!
- You can do it, too!





Universidad
Zaragoza

Artificial Intelligence (62651)

MAICE

